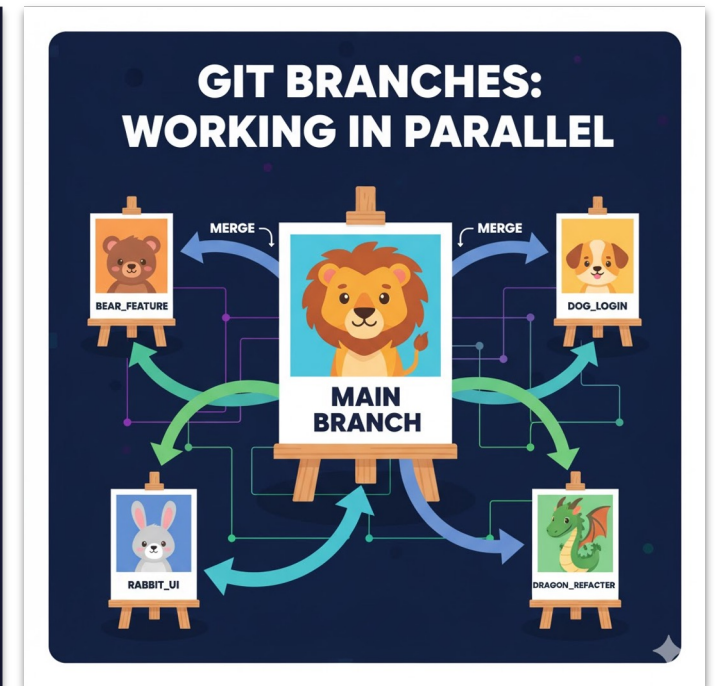
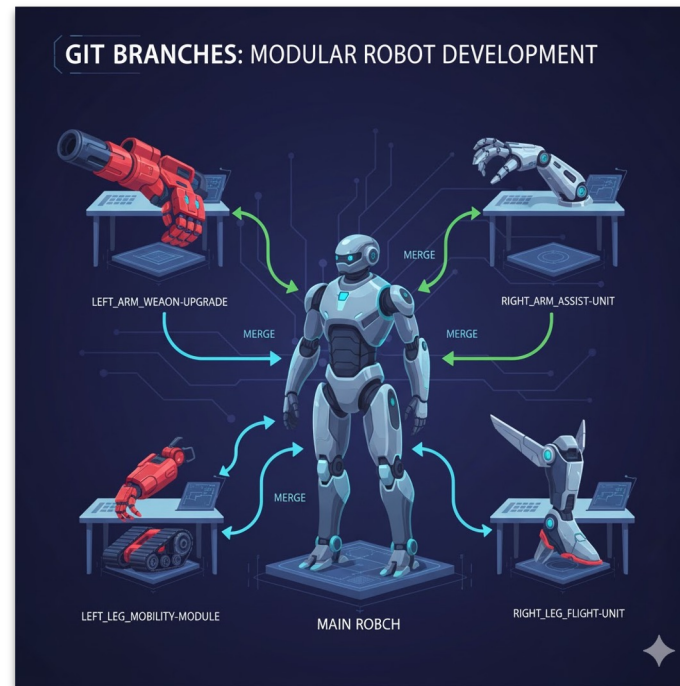


SESSION 8

브랜치

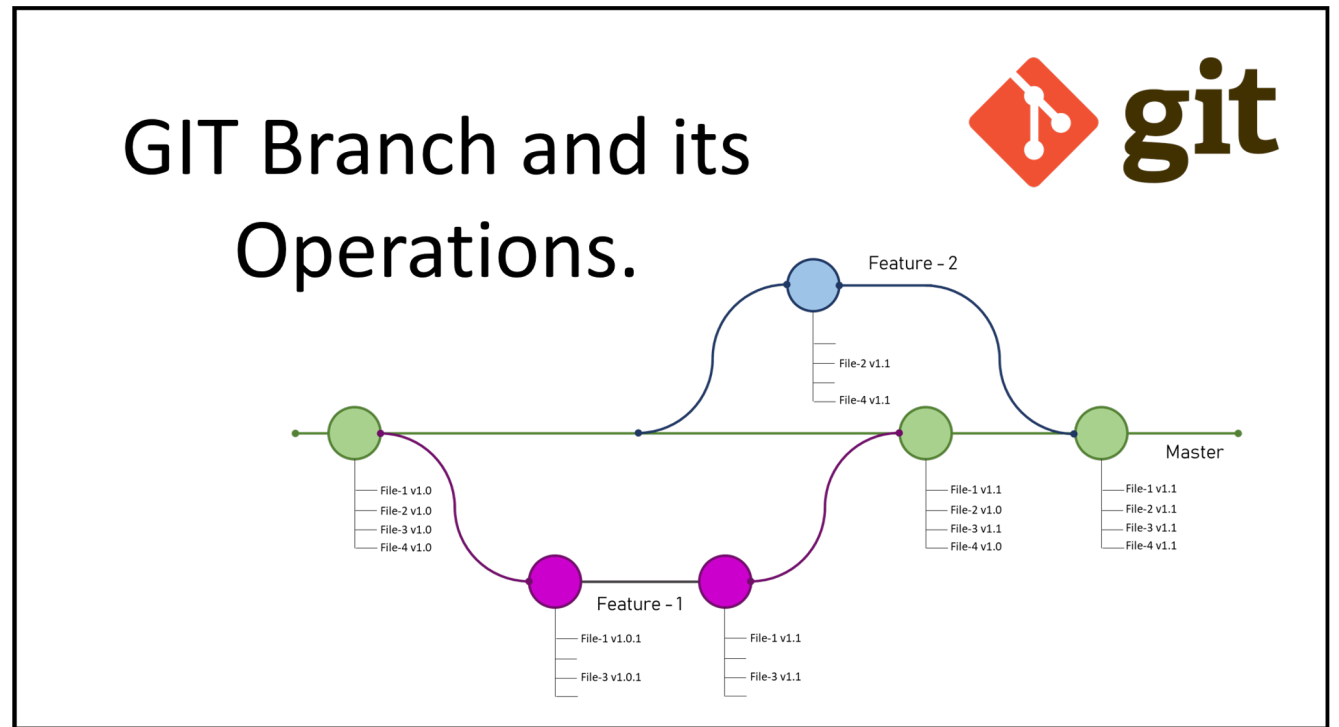
브랜치의 이해

로봇의 각 부위를 팀별로 담당하여 연구, 개발을 진행한다면...
대표 캐릭터를 다른 동물로 대체해보는건...



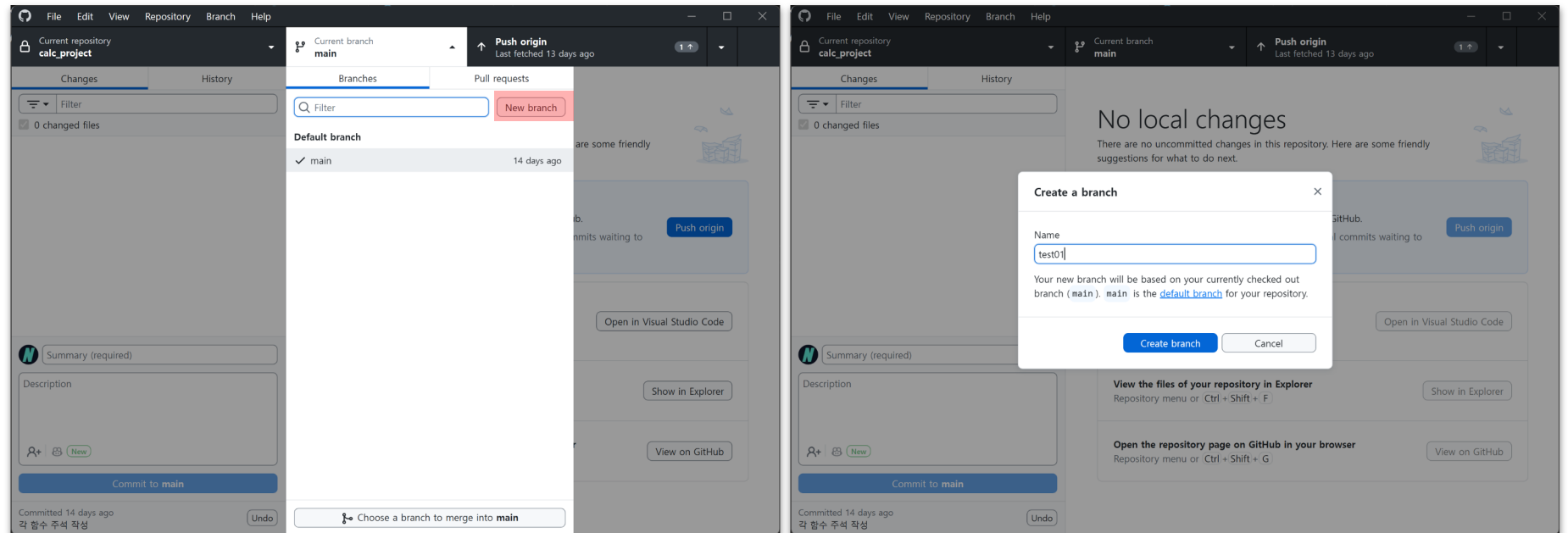
브랜치의 이해

제품화 되는 소스코드를 유지하면서 새로운 기능들을 개발하기 위해
동일 프로젝트 내에서도 여러가지 버전 필요



브랜치 활용하기

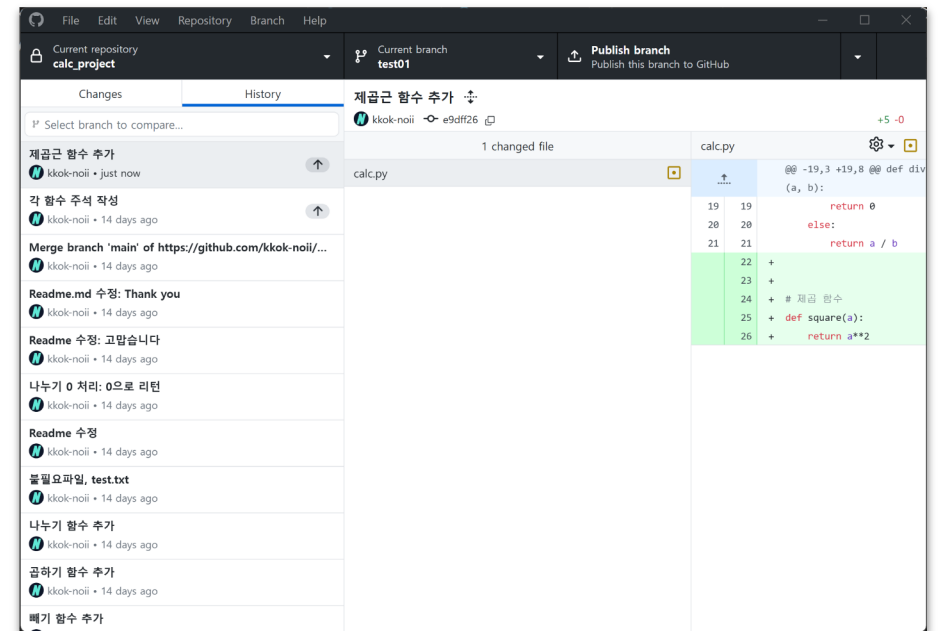
새로운 브랜치 test01을 만들어봅시다



브랜치 활용하기

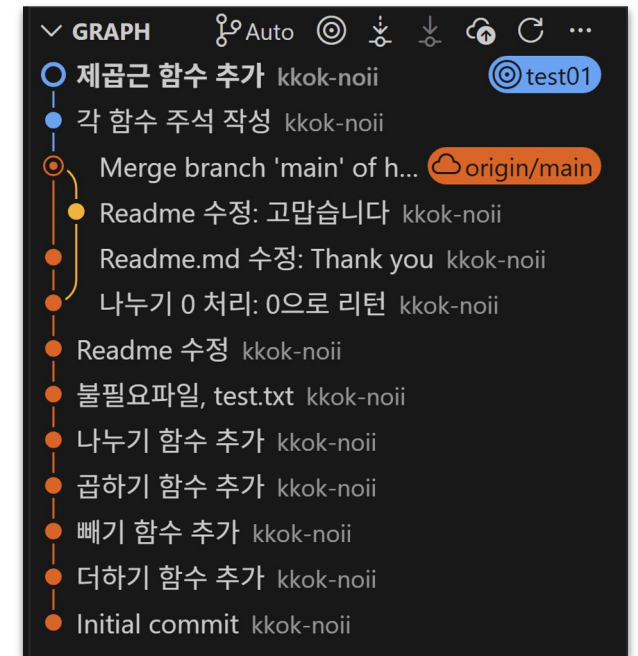
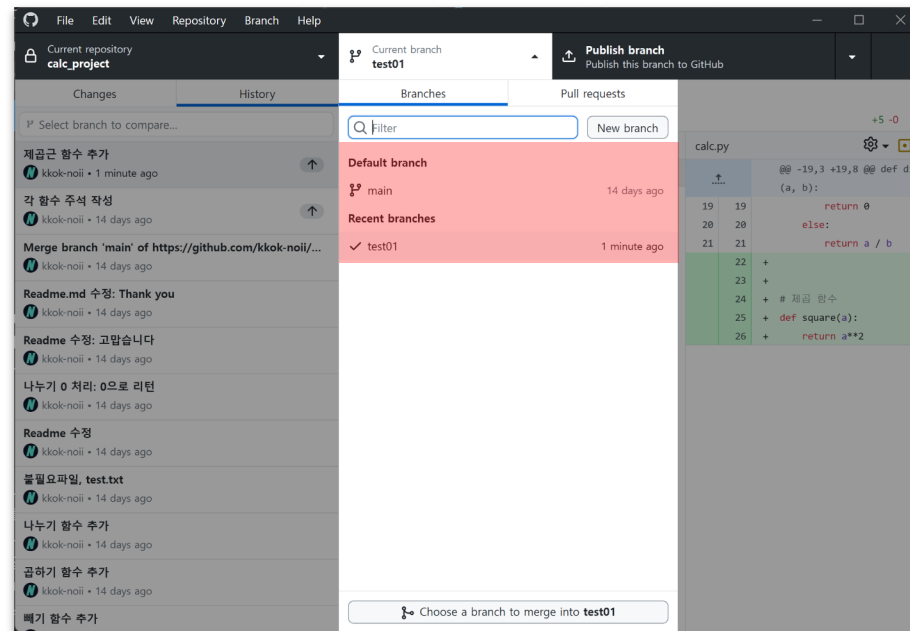
calc.py에 새로운 함수, **square**를 개발하고 Commit

```
24 # 제공 함수
25 def square(a):
26     return a**2
27
```



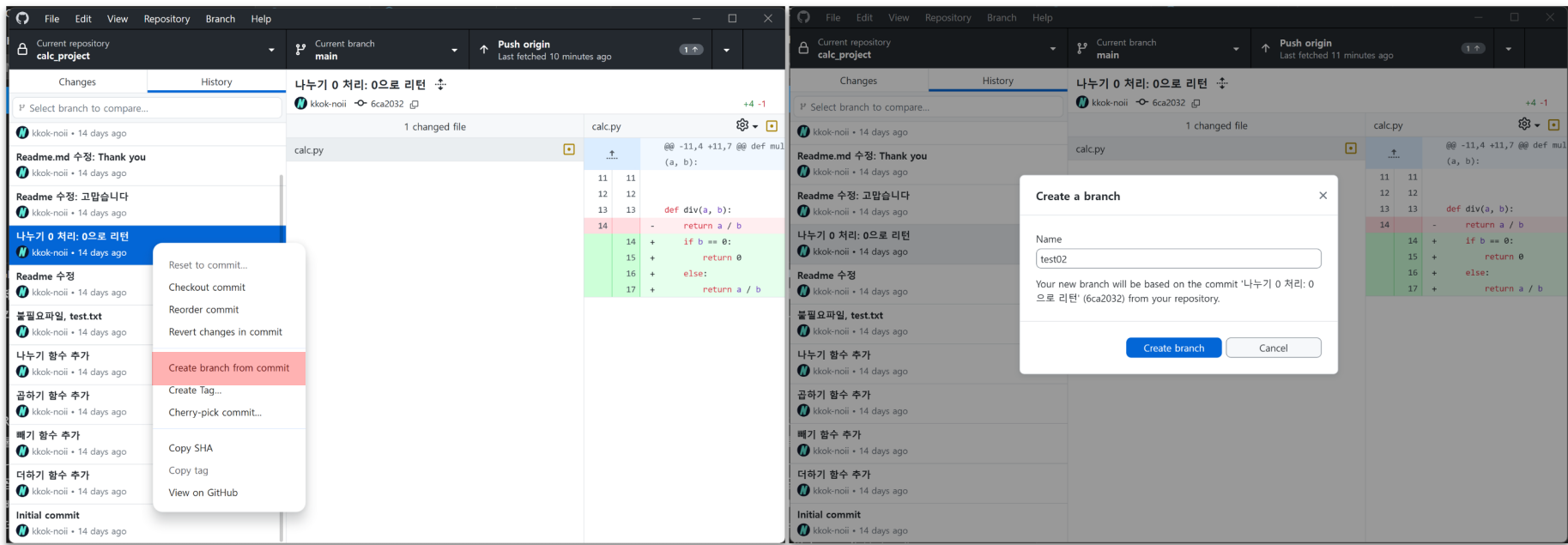
브랜치 활용하기

main 브랜치 또는 **test01** 브랜치를 선택해보고
소스코드를 확인해보세요



브랜치 활용하기

main 브랜치를 선택한 상태에서 특정 Commit 우클릭
Create branch from commit: test02 브랜치 만들기



참고

합치는 방식은 네 가지

방식

특징

Merge commit

기본값. 합친 기록이 커밋으로 남습니다.

Squash and merge

여러 커밋을 하나로 눌러 담습니다.

Rebase and merge

한 줄로 이어 붙여 히스토리가 깔끔합니다.

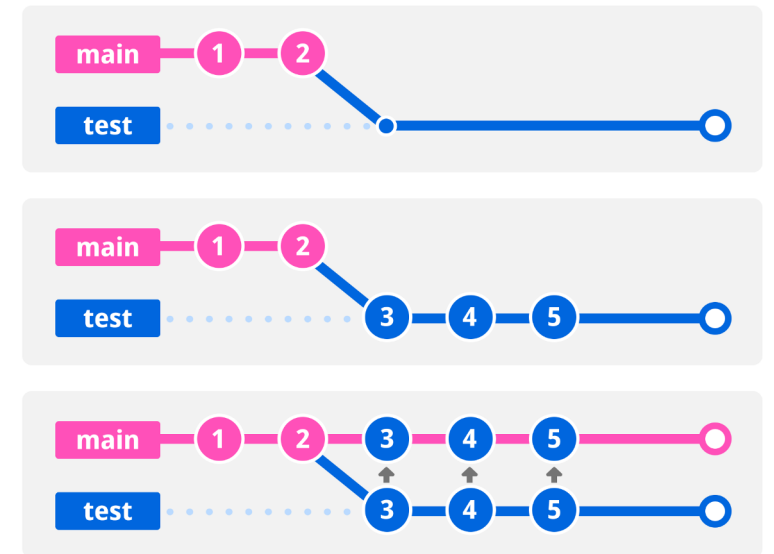
Fast-forward

갈라진 기록이 없을 때 그대로 앞으로 감깁니다.

Fast Forward

합치고자 하는 브랜치에 새로운 내용이 없으면, 그대로 이전

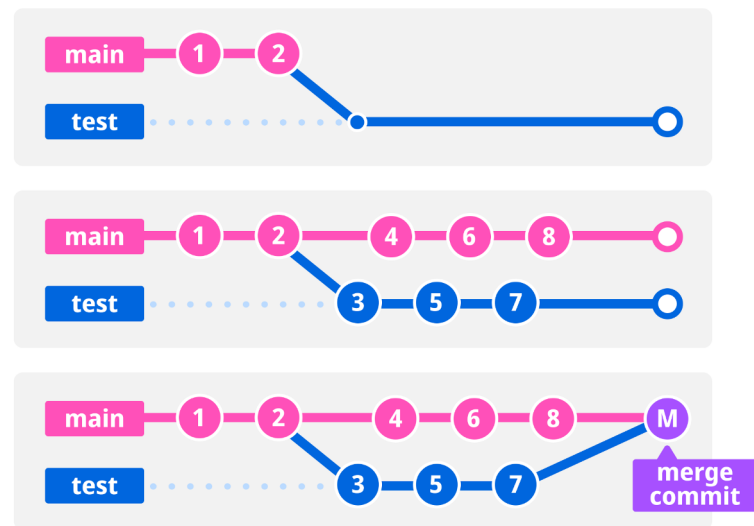
Merge (Fast Forward)



Create a merge commit

모든 이력을 남기며 Merge 됨을 새로운 Commit으로 남김

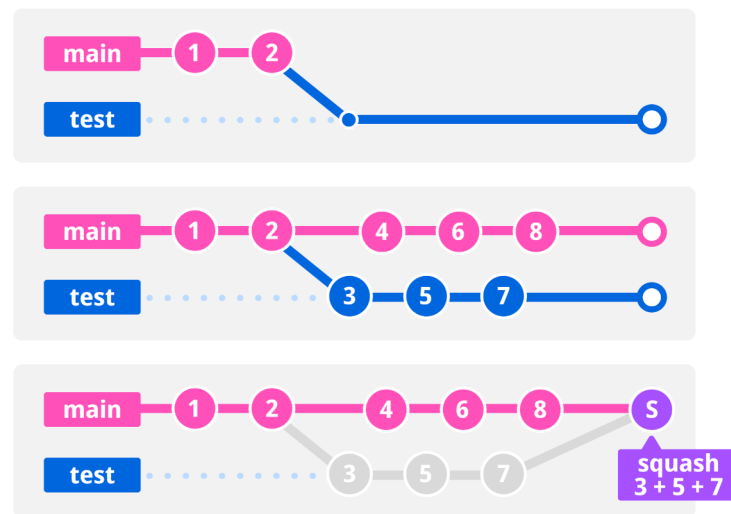
Merge (Recursive)



Squash & Merge

합치고자 하는 브랜치의 내용을 압축하여 하나의 Commit으로 남김

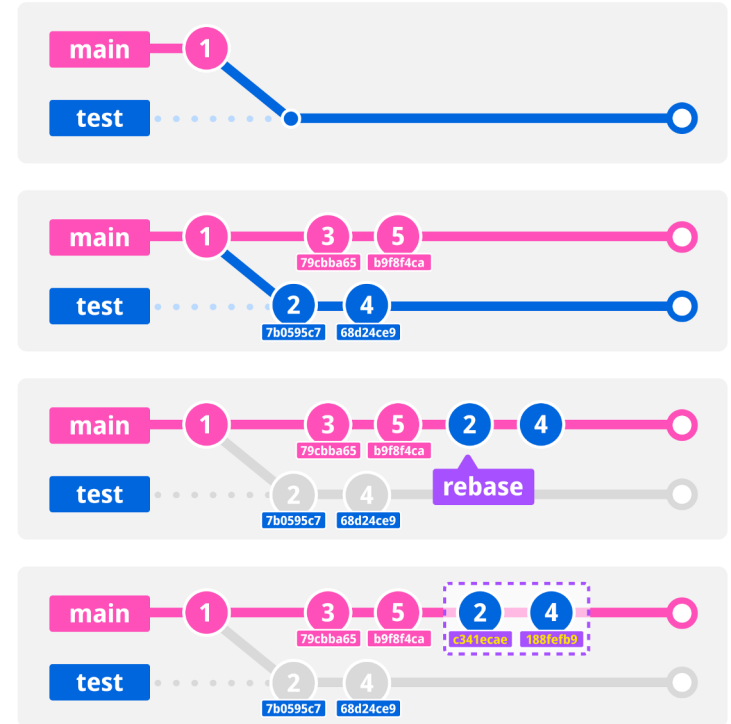
Squash & Merge



Rebase

합치고자 하는 브랜치의 Commit들을 Main Branch의 재 배열하여 Merge

Rebase & Merge



SESSION 9

Branch 전략

Branch 전략

프로젝트에서 여러 개발자가 협업할 때, 소스코드를 효율적으로 관리하기 위해 제안된 방법론 대표적인 3가지 (각 회사마다 이를 기반으로 다르게 운영할 수 있음)

- 01 Gitflow
- 02 Github flow
- 03 Gitlab flow

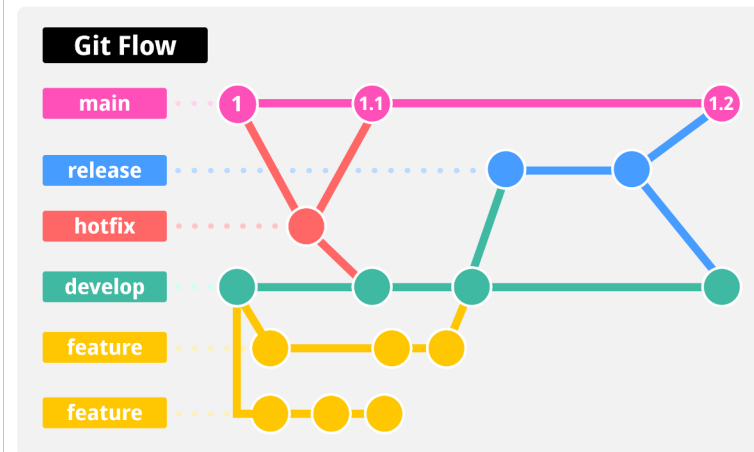
Gitflow

5개의 Branch를 활용하여 소스코드 관리

- 01 main: 기준이 되는 브랜치로 제품 최종 코드 관리
- 02 develop: 개발 브랜치로 개발자들이 이 브랜치를 기준으로 각자 작업한 기능들을 Merge
- 03 feature: 단위 기능을 개발하는 브랜치로 기능 개발이 완료되면 develop 브랜치로 Merge
- 04 release: 배포를 위해 master 브랜치로 보내기 전에 먼저 QA(품질검사)를 하기위한 브랜치
- 05 hotfix: master 브랜치로 배포를 했는데 버그가 생겼을 때 긴급 수정하는 브랜치

앱 서비스에 적합한 형태로 최종 결과물에 대한 버전 관리,
배포 전 제품을 한번 더 확인할 수 있는 상태가 필요할 때 적합

[우아한형제들 기술 블로그](#)



Github flow

main Branch가 최종 소스코드로써 항상 배포 가능한 상태

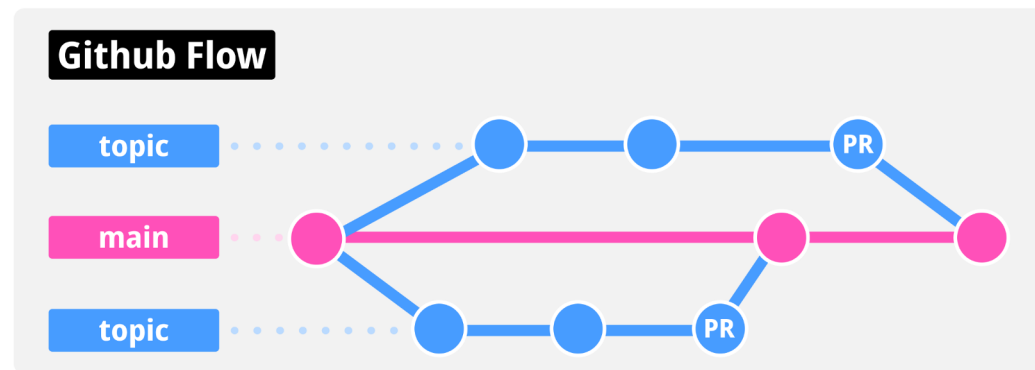
실제 작업은 다른 Branch에서 진행(버그 수정 or 기능 추가/변경: Branch 명명 규칙 없음)

작업이 완료되면 main Branch로 바로 합치기에

Code review가 필수적으로 필요함

웹 서비스에 적합한 형태로

수정된 부분이 상시 배포가 가능한 서비스에 적합한 형태



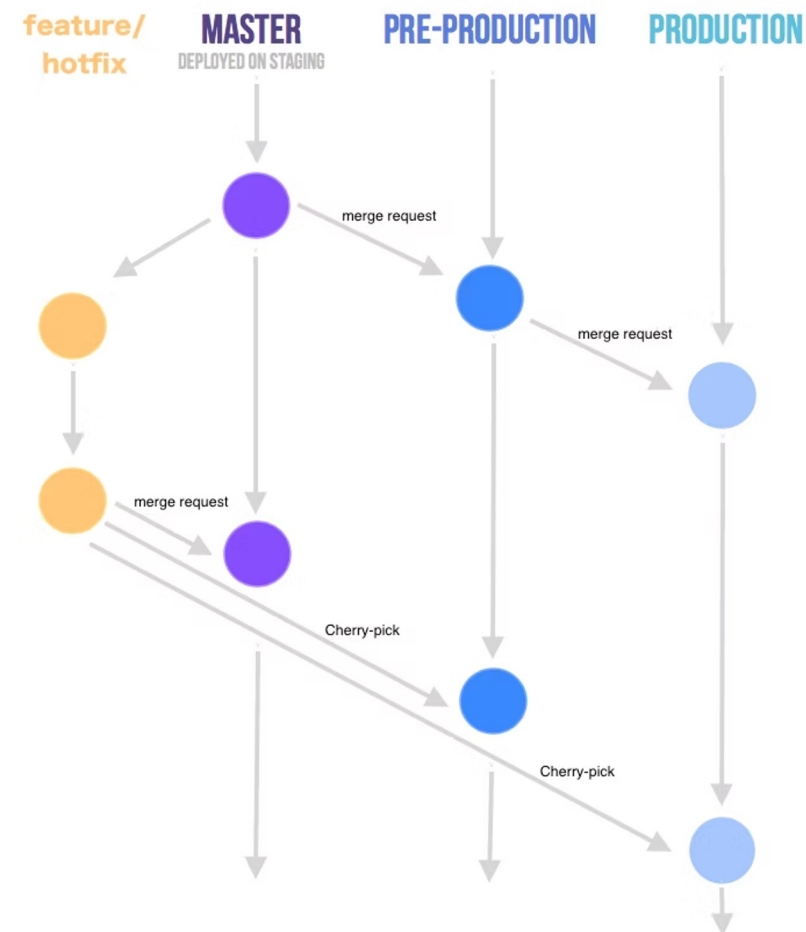
Github flow

gitflow와 github flow의 중간 느낌

master Branch를 중심으로 개발 진행(like github flow)

버그가 발생해도 master에서 진행

배포에 관해서만 PRE-PRODUCTION, PRODUCTION Branch 활용



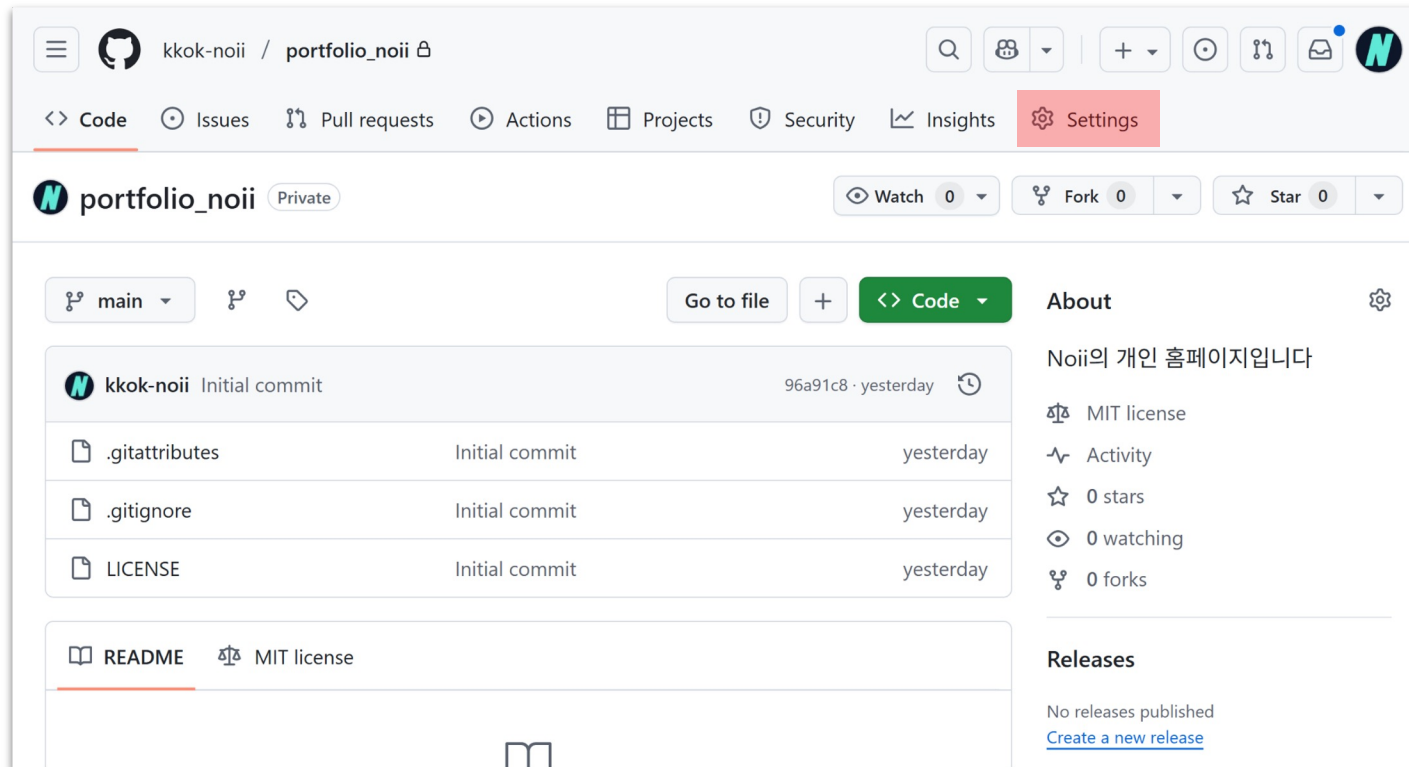
SESSION 10

Github 활용 협업 진행

협업하기 - Collaborator 초대

같이 작업을 진행할 Collaborator를 해당 Repository에 초대하겠습니다

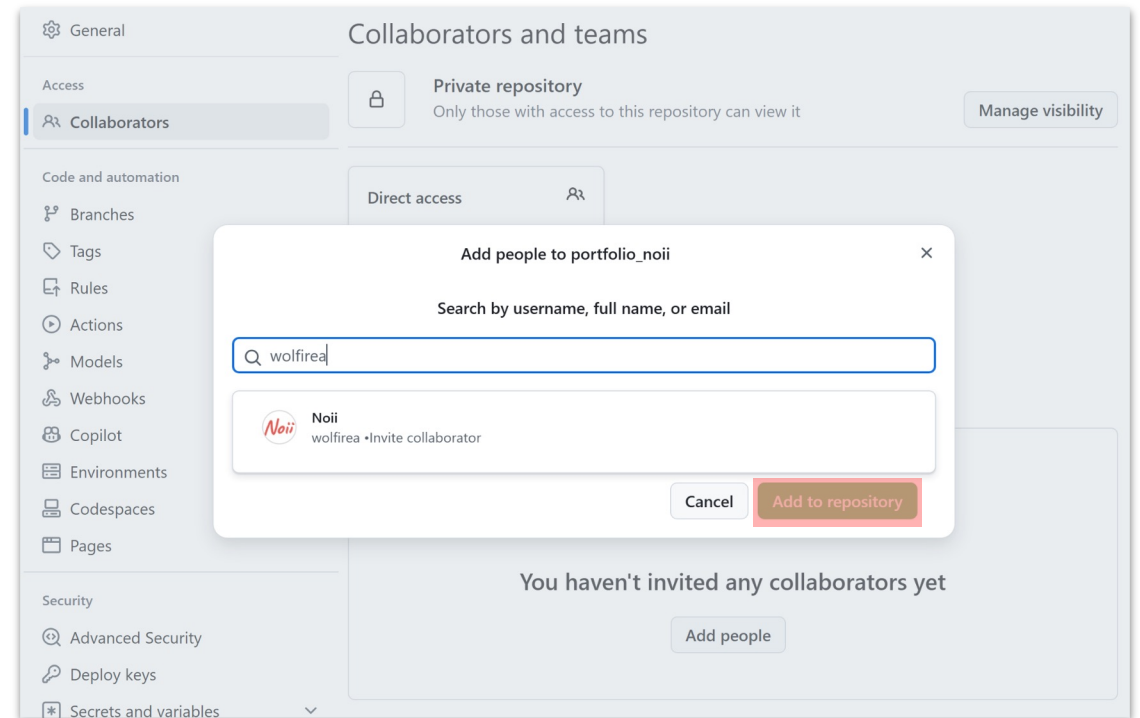
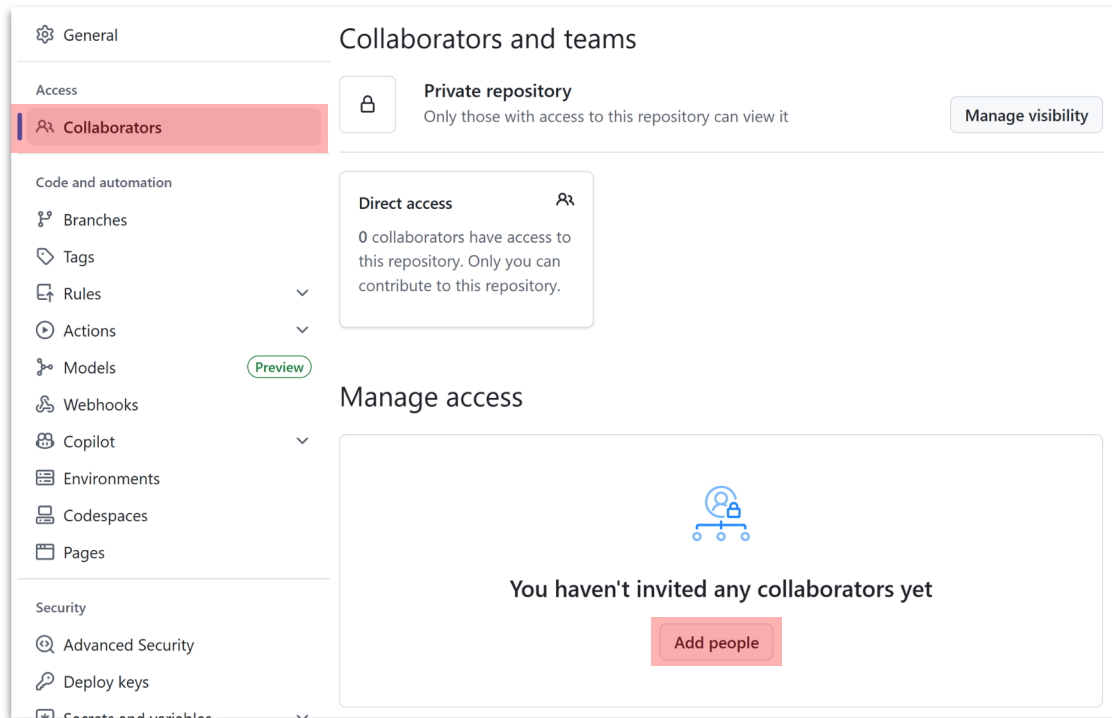
상단의 Settings로 들어가서...



협업하기 - Collaborator 초대

Collaborators > Add people

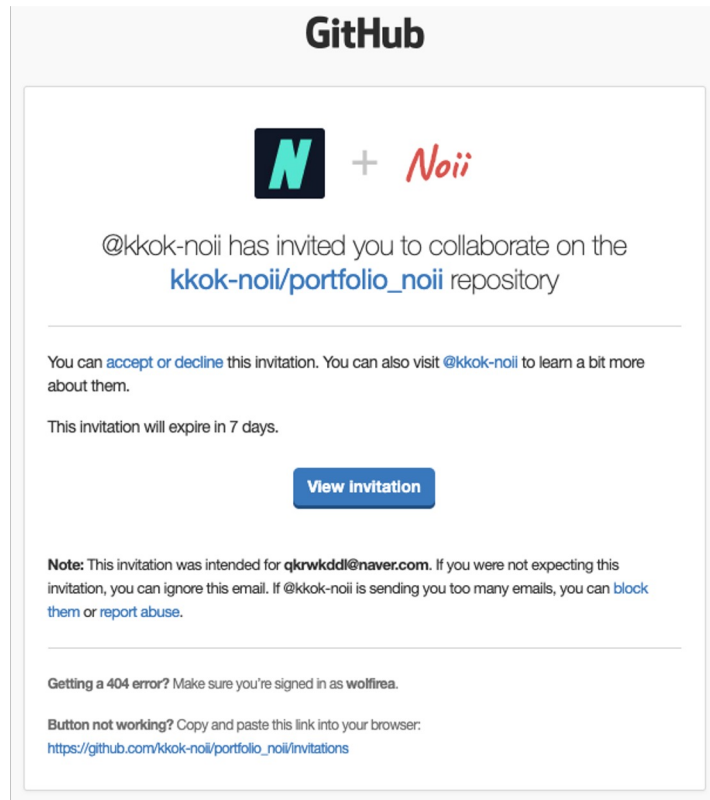
닉네임을 검색해서 추가(Add to repository) 해주세요 (서로 추가해주세요)



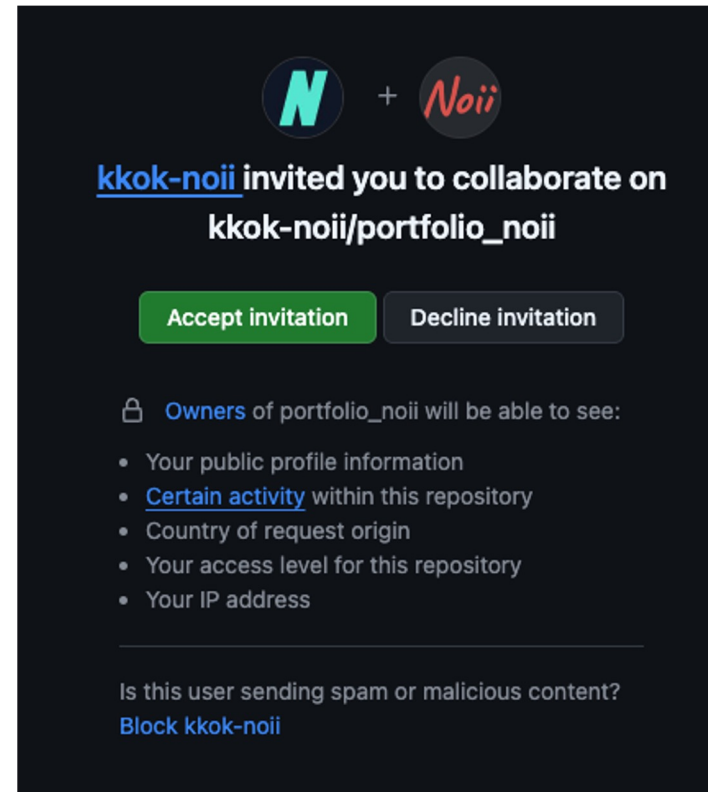
협업하기 - Collaborator 초대

초대가 되면, 받은사람에게 이메일로 초대 메일이 옵니다

View Invitation > Accept invitation 하면 해당 Repository에 접근할 수 있습니다



<이메일 내>



<Github 페이지 내>

협업하기 - Collaborator 초대

상대가 요청을 수락하면, 상태가 바뀝니다

Manage access Add people

☐ Select all Type ▾

Find a collaborator...

☐  **Noii**
Awaiting wolfirea's response Pending Invite  

↓

Manage access Add people

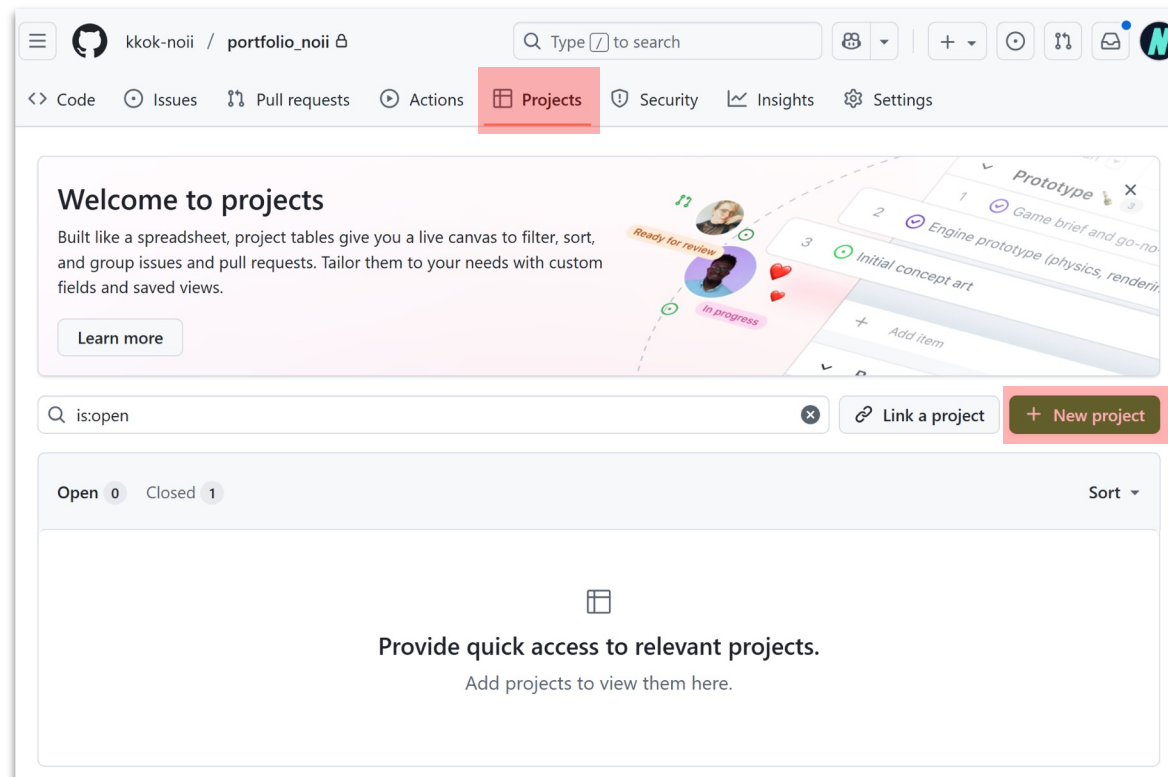
☐ Select all Type ▾

Find a collaborator...

☐  **Noii**
wolfirea • Collaborator 

협업하기 - 프로젝트

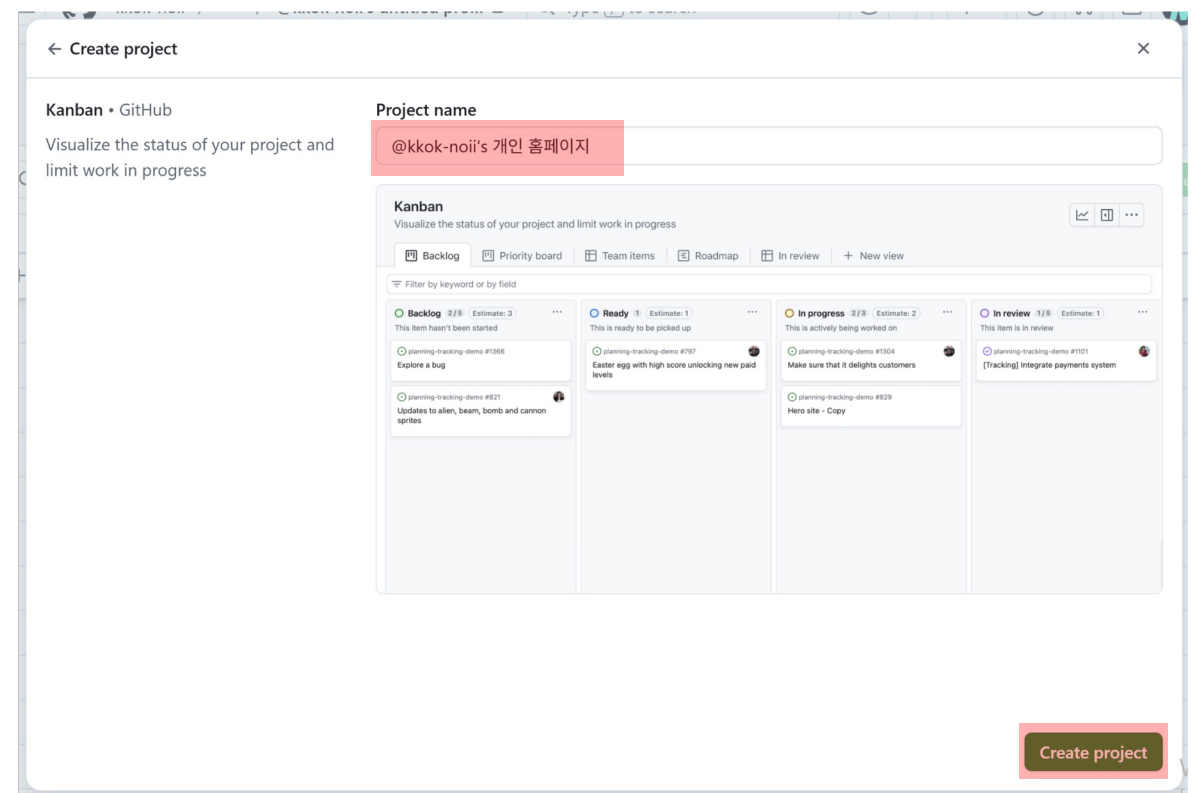
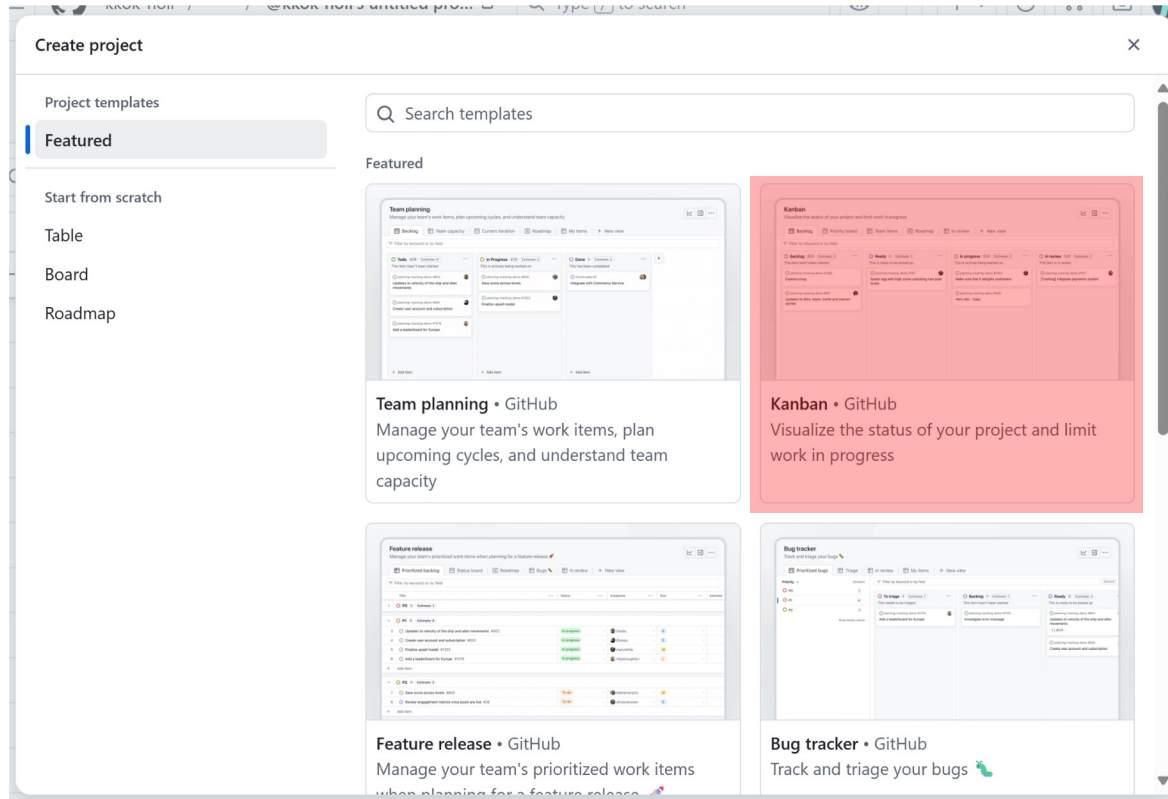
프로젝트는 협업하는 사람들끼리 하나의 목적을 달성하기 위해 작업의 우선순위, 상황 공유 등을 효율적으로 할 수 있도록 도와주는 도구



협업하기 - 프로젝트

새로운 프로젝트를 생성합니다

여러 유형이 있지만 Kanban 보드로 이름을 정하고 만들어 줍니다

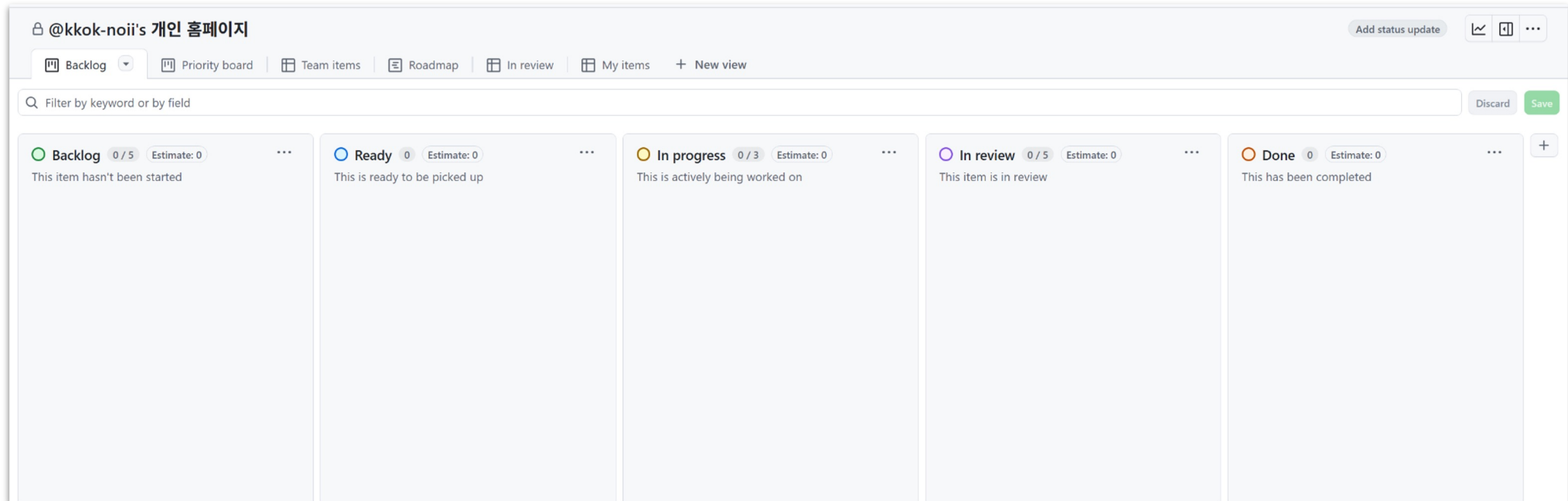


협업하기 - 프로젝트 생성

Kanban 보드

각 Issue들의 상태를 한눈에 볼 수 있는 보드판으로써 이슈를 드래그하여 이동하여 관리 가능

- Backlog: 살펴볼 예정
- In review: 작업 완료 후 검토 상태
- Ready: 다음 작업 예정
- Done: 작업 종료
- In progress: 작업 진행 준비 완료



협업하기 - 프로젝트 생성

해당 프로젝트에도 Collaborator를 추가해줍니다

The image shows a Notion workspace interface. At the top, the breadcrumb navigation reads "kok-noii / ... / @kok-noii's 개인 홈페이지". Below this, the page title is "@kok-noii's 개인 홈페이지". The main content area displays a "Backlog" view with a single item: "portfolio_noii #5" with the description "메인페이지 사진 추가하기". A sidebar menu on the right contains options: "Workflows", "Archived items", "Settings" (highlighted in red), and "Make a copy".

Overlaid on the bottom right is the "Settings" modal. The left sidebar of the modal lists "Project settings" and "Manage access" (highlighted in red). The main content of the modal is divided into two sections:

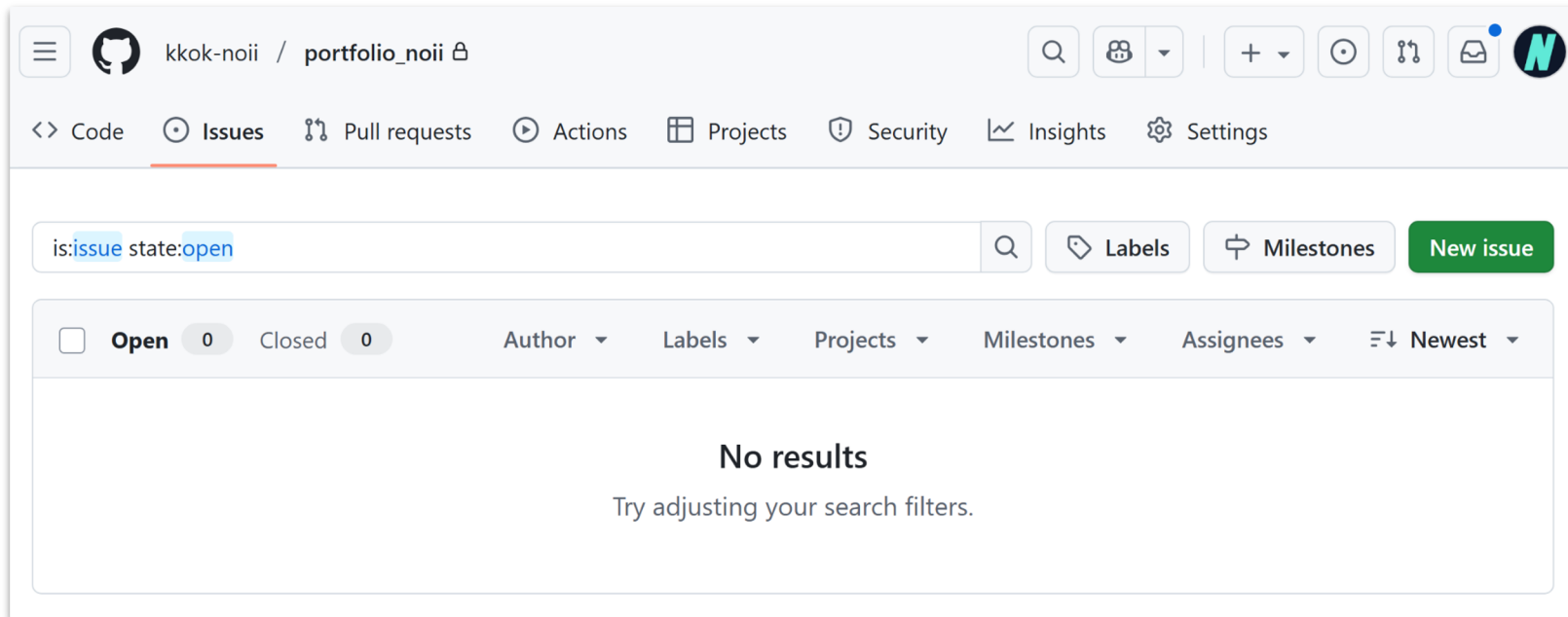
- Who has access**: Shows the project is a "Private project" and includes a "Manage" button.
- Invite collaborators**: Features a search bar with the text "Search by username", a dropdown menu for "Role" set to "Write", and a green "Invite" button.

In the search bar of the "Invite collaborators" section, the user "wolfire" is listed as a collaborator.

협업하기 - 이슈

이슈는 해당 프로젝트에서 해야하는 모든 일, 하나하나를 가리킵니다

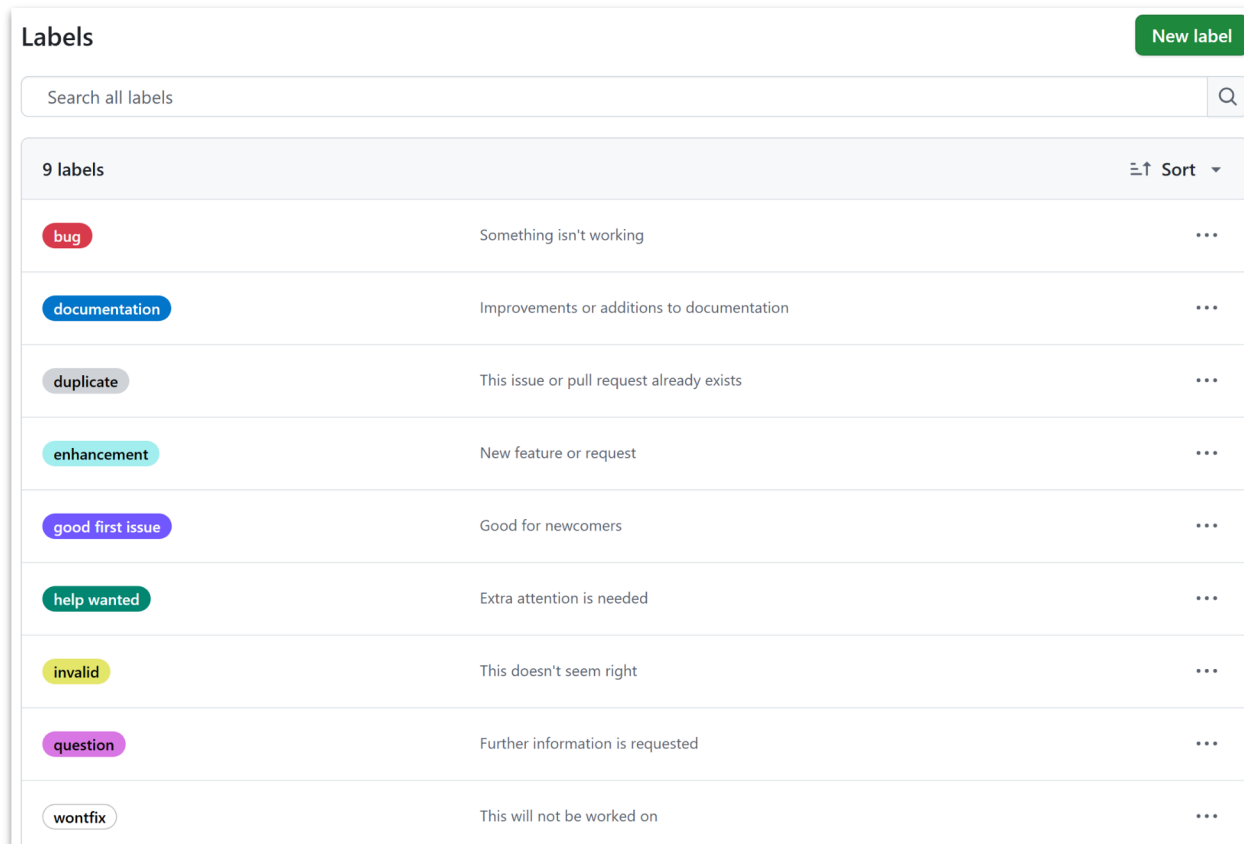
버그 수정, 문서화, 기능개발 등등



협업하기 - 이슈

이슈를 종류별로 구별하기 위한 Label

회사, 팀마다 다르게 관리할 수 있습니다



협업하기 - 이슈 생성

새로운 이슈는 마치 게시글 처럼 작성할 수 있습니다

이슈는 해당 일이 시작되고 끝날 때까지의 모든 과정을 기록합니다

 Create new issue

Add a title *

프로젝트 패키지 설정

Add a description

WritePreview

H B I        

필요 패키지를 설치하고 requirements.txt를 작성합니다
- 필요패키지: streamlit

 Add Files  Write with Copilot

☐ Create more

Cancel

Create  

Assignees

 kkok-noii

Labels

enhancement

Projects

@kkok-noii's 개인 홈페이지

Milestone

No milestone

협업하기 - 이슈 생성

새로운 이슈는 마치 게시물처럼 작성할 수 있습니다

이슈는 해당 일이 시작되고 끝날 때까지의 모든 과정을 기록합니다

프로젝트 패키지 설정 #1

EditNew issue

Open

kkok-noii opened 1 minute ago

필요 패키지를 설치하고 requirements.txt를 작성합니다

- 필요패키지: streamlit

Create sub-issue

kkok-noii self-assigned this 1 minute ago

kkok-noii added enhancement 1 minute ago

kkok-noii moved this to Ready in @kkok-noii's 개인 홈페이지 now

Add a comment

WritePreview

H B I

Use Markdown to format your comment

Assignees

kkok-noii

Labels

enhancement

Projects

@kkok-noii's 개인 홈페이지

StatusReady

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

협업하기 - 이슈 생성

다음에 해당하는 이슈를 만들어보세요

1. 제공 함수 만들기 (새로운 기능)
2. 나머지 함수 만들기 (새로운 기능)
3. README 수정하기 (문서)

협업하기 - PR

main 브랜치가 아닌 다른 브랜치를 Push 하면 Compare & pull request 버튼이 활성화 됩니다

위쪽 Pull requests 메뉴에서도 접근이 가능합니다

The screenshot shows the GitHub interface for a repository named 'portfolio_noii' by user 'kkok-noii'. The top navigation bar includes links for Code, Issues (5), Pull requests (highlighted in red), Actions, Projects (2), Security, Insights, and Settings. Below the navigation bar, the repository name 'portfolio_noii' is shown as 'Private'. To the right of the name are buttons for Watch (0), Fork (0), and Star (0). A yellow banner at the top of the repository content area states 'issue1 had recent pushes 32 minutes ago' and features a 'Compare & pull request' button. Below this banner, the repository's branch structure is shown: 'main' (selected), '2 Branches', and '0 Tags'. A search bar 'Go to file' is present. The commit history table shows a single commit by 'kkok-noii' with the message 'Initial commit', hash '96a91c8', and timestamp 'yesterday', with '1 Commit' in total. The file list includes '.gitattributes', '.gitignore', and 'LICENSE', all marked as 'Initial commit' and 'yesterday'. On the right side, the 'About' section describes the repository as 'Noii의 개인 홈페이지입니다' and lists 'MIT license', '0 stars', '0 watching', and '0 forks'. The 'Releases' section states 'No releases published' with a link to 'Create a new release'. The 'Packages' section states 'No packages published' with a link to 'Publish your first package'.

kkok-noii / portfolio_noii

Type / to search

<> Code Issues 5 Pull requests Actions Projects 2 Security Insights Settings

portfolio_noii Private

Watch 0 Fork 0 Star 0

issue1 had recent pushes 32 minutes ago Compare & pull request

main 2 Branches 0 Tags Go to file + <> Code

kkok-noii	Initial commit	96a91c8 · yesterday	1 Commit
.gitattributes	Initial commit	yesterday	
.gitignore	Initial commit	yesterday	
LICENSE	Initial commit	yesterday	

README MIT license

Add a README

About

Noii의 개인 홈페이지입니다

MIT license

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

협업하기 - PR

Pull request도 작성합니다

하나의 Issue로써 프로젝트에서 관리가 돼요

Developer의 서태채서 가이 거트하 도르에게 아리 스 이습니다

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main ← compare: issue1

Add a title

#1 프로젝트 패키지 설정

Add a description

Write Preview H B I `< >` [@](#) [🔗](#) [📎](#) [↩](#) [📄](#)

#1

- 가상환경 설정폴더(venv)를 ignore에 추가했습니다
- Streamlit 설치 후 requirements.txt를 작성했습니다

Markdown is supported Paste, drop, or click to add files

Reviewers

No reviews

Assignees

kkok-noii

Labels

enhancement

Projects

@kkok-noii's 개인 홈페이지

Milestone

No milestone

Development

Use [closing keywords](#) in the description to automatically close issues

[Helpful resources](#)

Create pull request

2 commits 2 files changed 1 contributor

Commits on Sep 17, 2025

ignore 리스트에 venv 추가
kkok-noii committed 1 hour ago

requirements.txt 작성 완료
kkok-noii committed 53 minutes ago

Showing 2 changed files with 4 additions and 1 deletion.

Split Unified

5 `.gitignore`

```
@@ -178,4 +178,7 @@ cython_debug/
178 178 # exclude from AI features like autocomplete and code analysis. Recommended for sensitive data
179 179 # refer to https://docs.cursor.com/context/ignore-files
180 180 .cursorignore
181 - .cursorindexingignore
181 + .cursorindexingignore
182 +
183 + # noii
184 + venv/
```

BIN +1.32 KB requirements.txt

Binary file not shown.

협업하기 - PR

충돌이 없는지 체크 후, 문제 없다면
Merge를 진행할 수 있습니다

Merge는 앞서 본 방식 중 선택하여 진행 가능합니다

Merge pull request

✓ **Create a merge commit**
All commits from this branch will be added to the base branch via a merge commit.

Squash and merge
The 2 commits from this branch will be combined into one commit in the base branch.

Rebase and merge
The 2 commits from this branch will be rebased and added to the base branch.

#1 프로젝트 패키지 설정 #6
Open kkok-noii wants to merge 2 commits into `main` from `issue1`

Conversation 0 Commits 2 Checks 0 Files changed 2

kkok-noii commented now

#1

- 가상환경 설정폴더(venv)를 ignore에 추가했습니다
- Streamlit 설치 후 requirements.txt를 작성했습니다

kkok-noii added 2 commits 1 hour ago

- ignore 리스트에 venv 추가 `7f0a74e`
- requirements.txt 작성 완료 `5889b6e`

kkok-noii self-assigned this now

kkok-noii added the `enhancement` label now

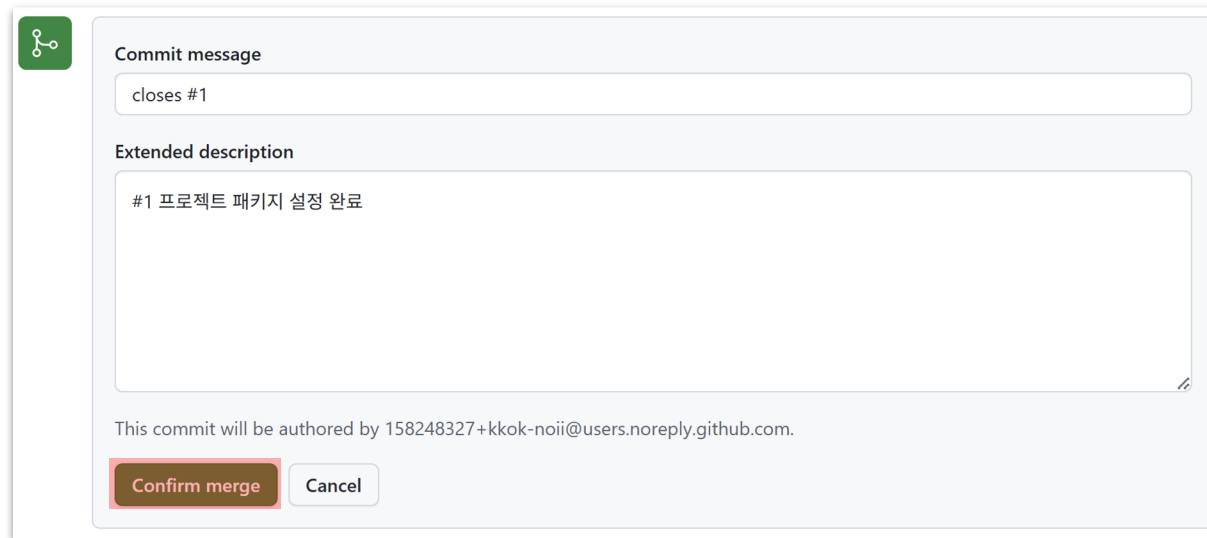
✓ **No conflicts with base branch**
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

협업하기 - PR

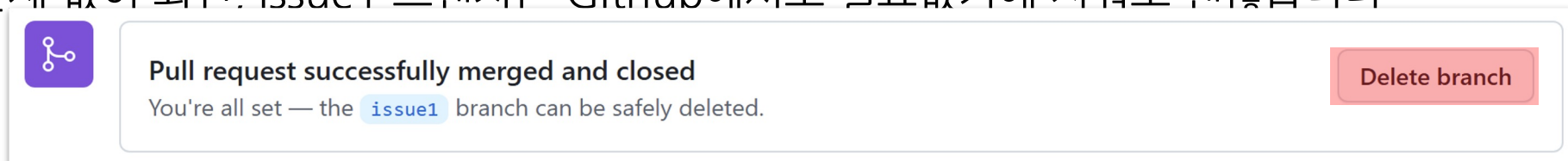
merge commit에 남길 message를 작성합니다

fixes #123, closes #123, resolves #123 와 같은 형태로 남기면 Issue가 자동으로 Closed 상태로 바뀝니다



The screenshot shows the 'Commit message' dialog box in GitHub. It has a green icon with a branch symbol in the top left corner. The dialog is divided into two main sections: 'Commit message' and 'Extended description'. The 'Commit message' section contains a text input field with the text 'closes #1'. The 'Extended description' section contains a larger text area with the text '#1 프로젝트 패키지 설정 완료'. Below these sections, there is a line of text stating 'This commit will be authored by 158248327+kkok-noii@users.noreply.github.com.' At the bottom of the dialog, there are two buttons: 'Confirm merge' (highlighted in orange) and 'Cancel' (a standard gray button).

Merge가 문제 없이 되면, issue1 브랜치는 Github에서도 필요없기에 지워도 괜찮습니다



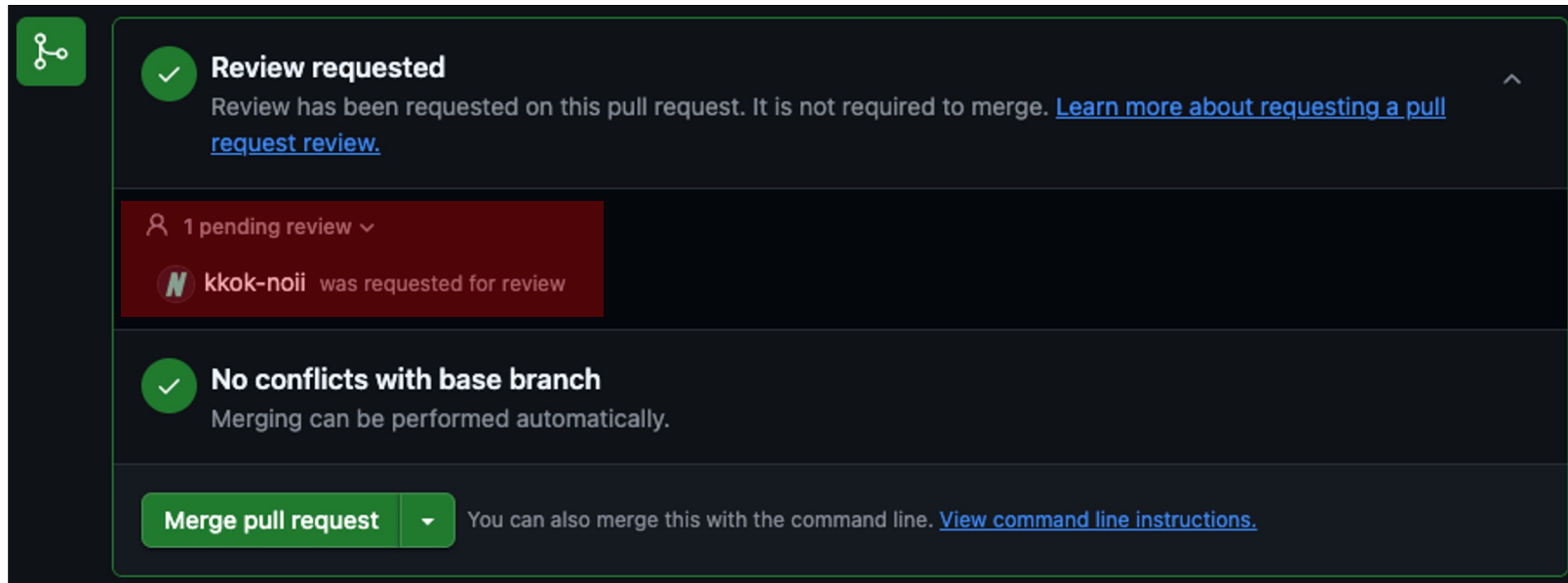
The screenshot shows a notification banner from GitHub. On the left, there is a purple icon with a branch symbol. The main text of the banner reads 'Pull request successfully merged and closed' in bold, followed by 'You're all set — the `issue1` branch can be safely deleted.' On the right side of the banner, there is a red button labeled 'Delete branch'.

7. Pull request 작성하기 - 리뷰어를 꼭 추가해주세요!

GitHub Community Guidelines

협업하기 - 리뷰

리뷰어를 추가하고 생성하면 다음과 같이 Review를 기다린다고 표기됩니다



협업하기 - 리뷰

현재 Repository는 Pull request Merge에 대해서 Review를 권장하고 있어,

Review가 완료되지 않아도 Merge할 수 있어요

Repository > Settings > Branches > Add branch ruleset에서 설정할 수 있습니다

지금은 Pass 😊

☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Hide additional settings ^

Required approvals

2 ▾

The number of approving reviews that are required before a pull request can be merged.

☐ **Dismiss stale pull request approvals when new commits are pushed**
New, reviewable commits pushed will dismiss previous pull request review approvals.

☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☐ **Require conversation resolution before merging**
All conversations on code must be resolved before a pull request can be merged.

☐ **Automatically request Copilot code review**
Request Copilot code review for new pull requests automatically if the author has access to Copilot code review.

Allowed merge methods

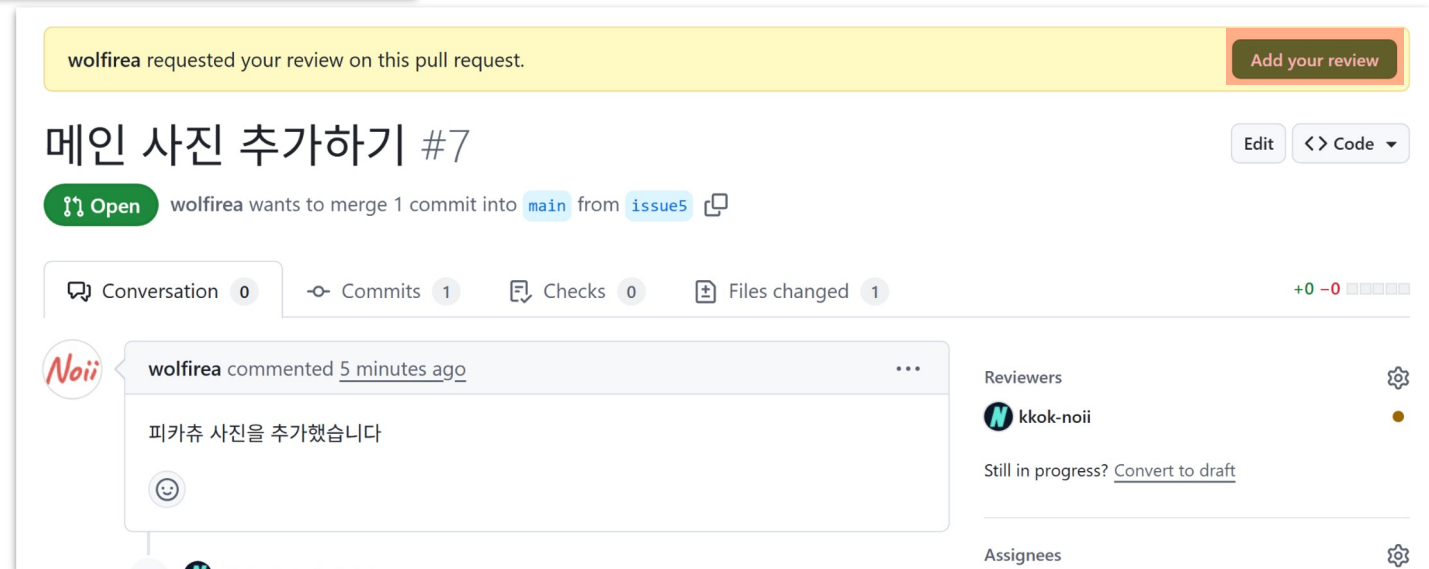
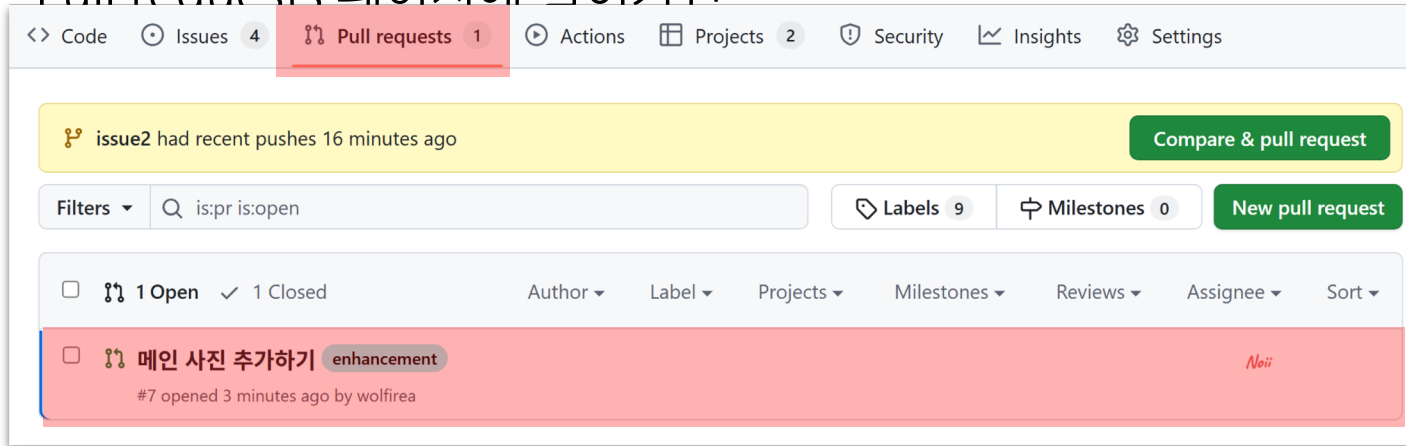
Merge, Squash, Rebase ▾

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled.

협업하기 - 리뷰

리뷰 요청이 들어오면 메일과 Github Desktop으로 알림이 옵니다

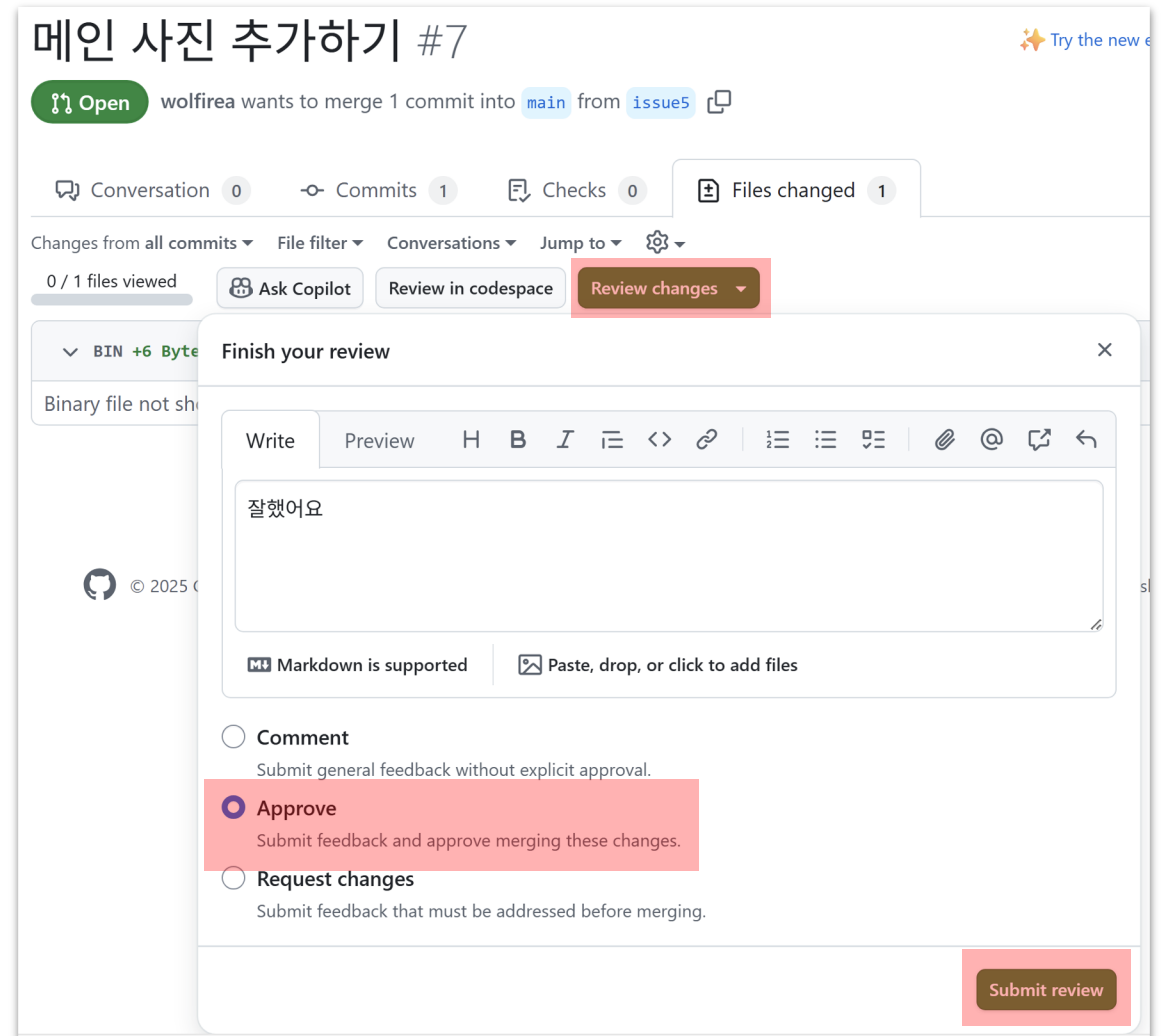
Pull requests 페이지에 들어가면 ...



협업하기 - 리뷰

해당 Pull request에 변경된 점을 확인하고

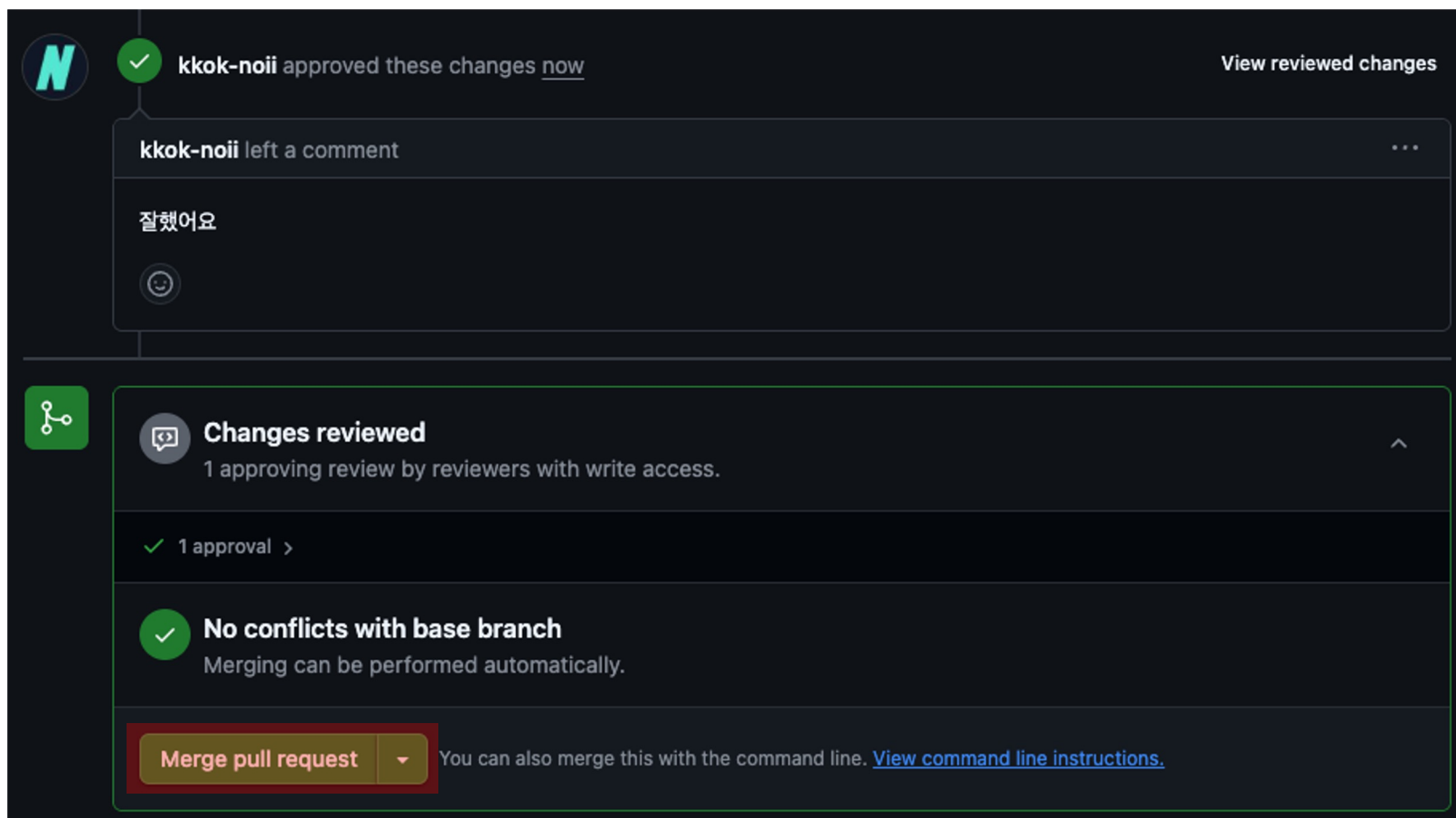
Comment를 남기거나 승인할 수 있어요



협업하기 - 리뷰

Review가 승인되면 작성자에게도 알람이 옵니다

이후에 Merge를 해주면 완료!



The screenshot displays a GitHub pull request interface. At the top, a notification shows that user 'kkok-noii' has approved the changes, with a green checkmark icon and the text 'kkok-noii approved these changes now'. To the right of this notification is a link that says 'View reviewed changes'. Below the notification, a comment box shows that 'kkok-noii' left a comment in Korean: '잘했어요' (Good job). Below the comment box, there is a section titled 'Changes reviewed' with a green checkmark icon, indicating that the changes have been reviewed. This section states '1 approving review by reviewers with write access.' and '1 approval >'. Below this, there is a green checkmark icon and the text 'No conflicts with base branch', indicating that the pull request can be merged automatically. At the bottom, there is a red button labeled 'Merge pull request' and a link that says 'You can also merge this with the command line. View command line instructions.'

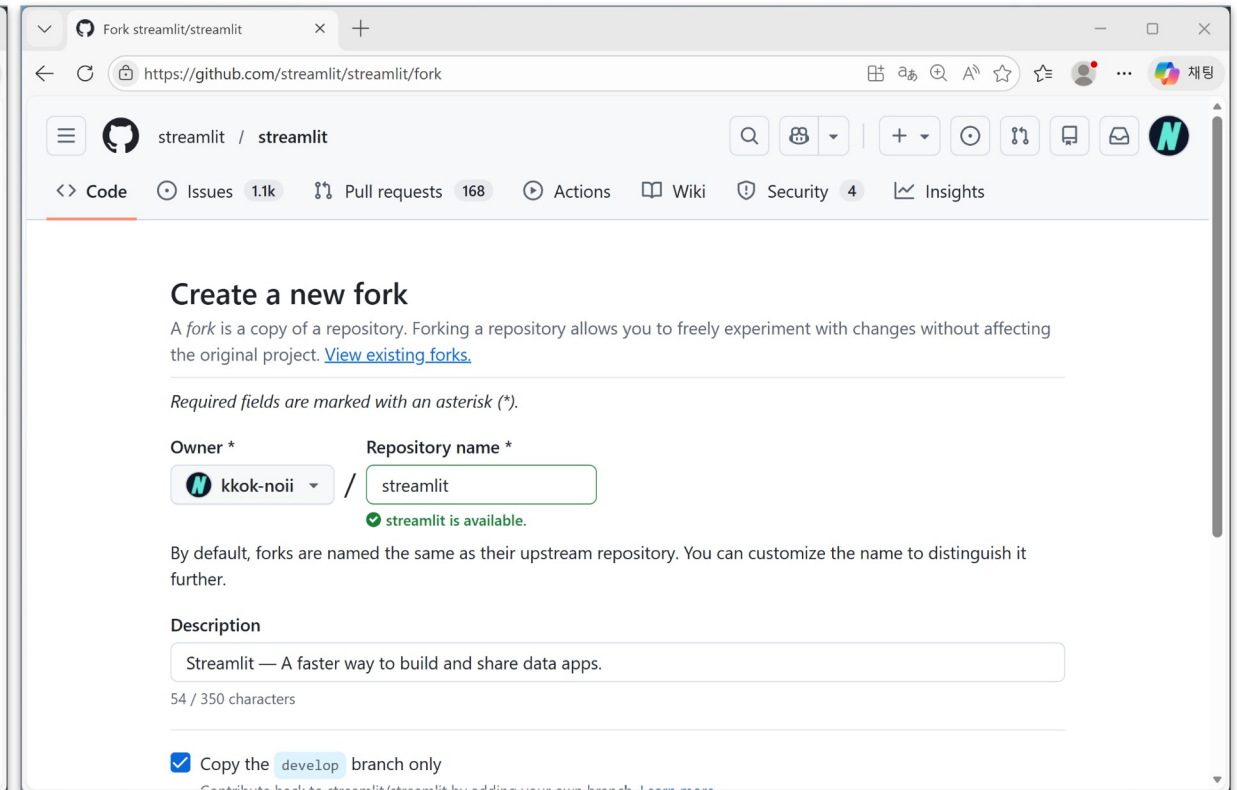
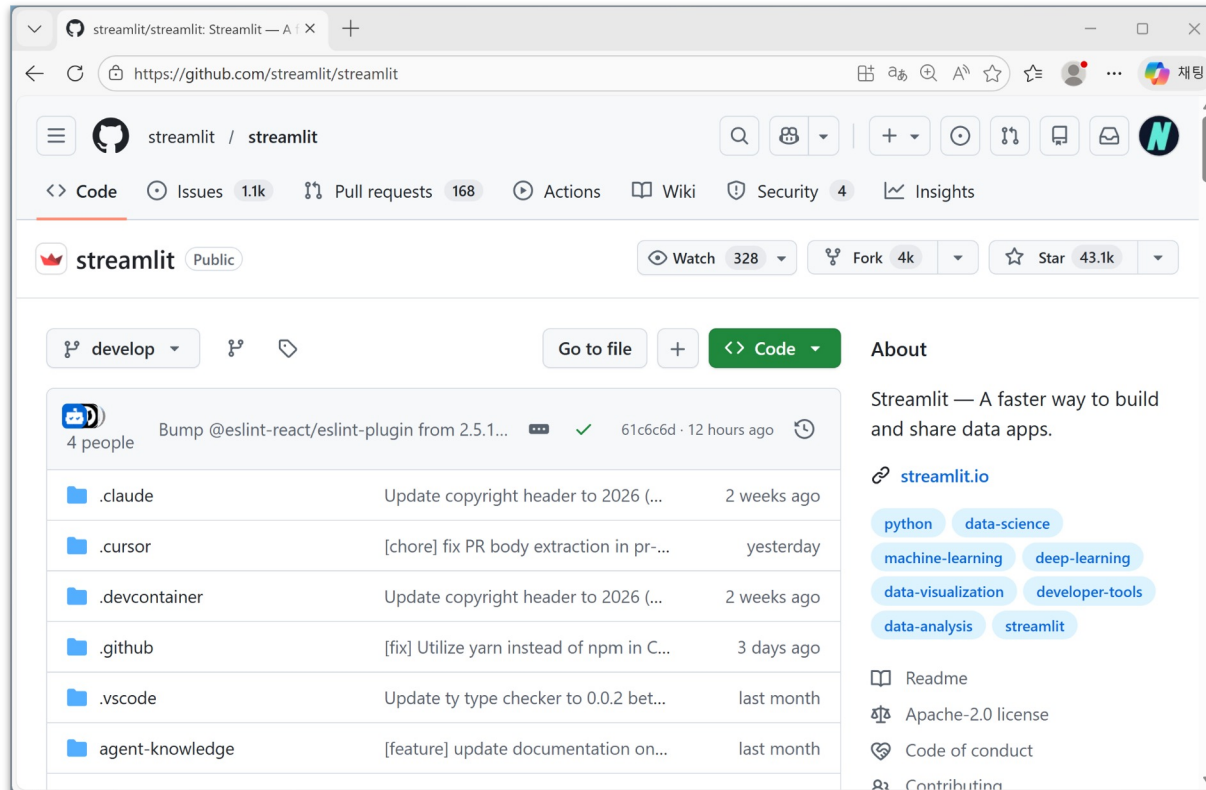
SESSION 11

Github Fork

Fork

Github 내에서 다른 Repository를 복사해올 수 있습니다

내 계정으로 가져와 작업 후, PR 가능!!



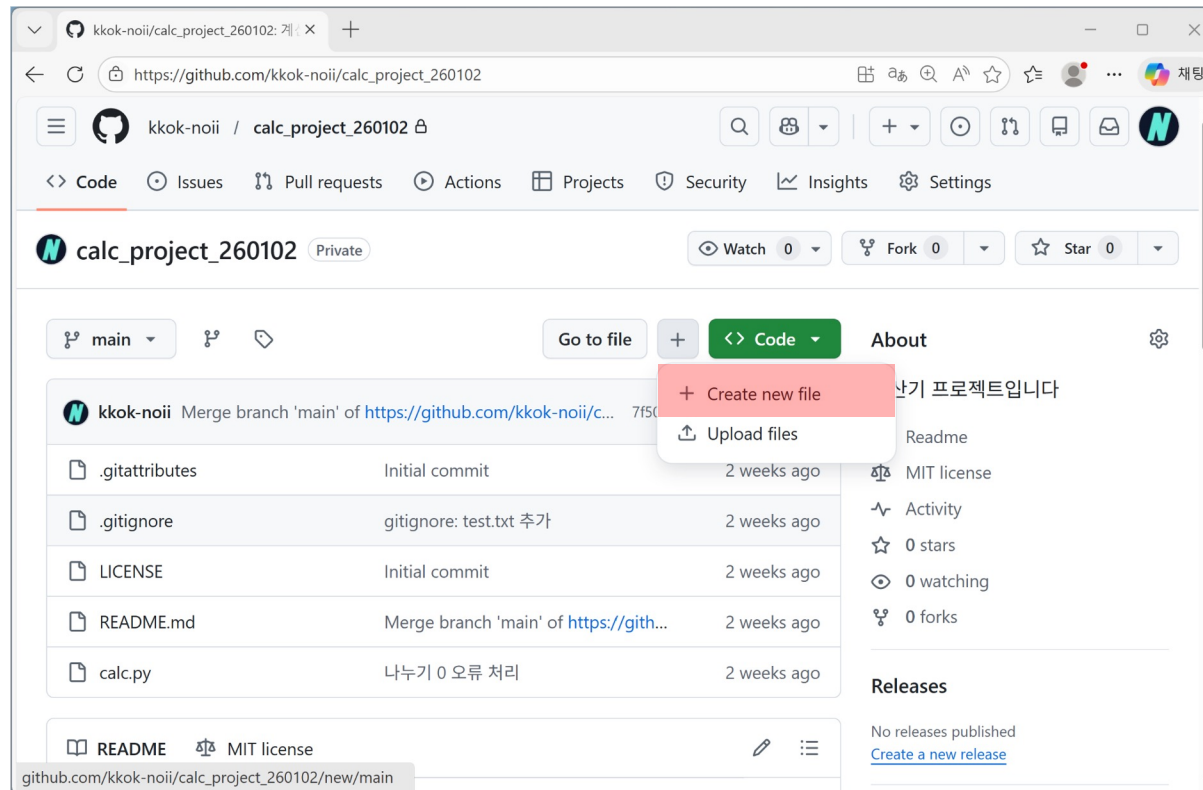
SESSION 12

Github Action

Action

Github repository에 코드가 올라올 때, 특정 행동을 하도록 지정할 수 있습니다

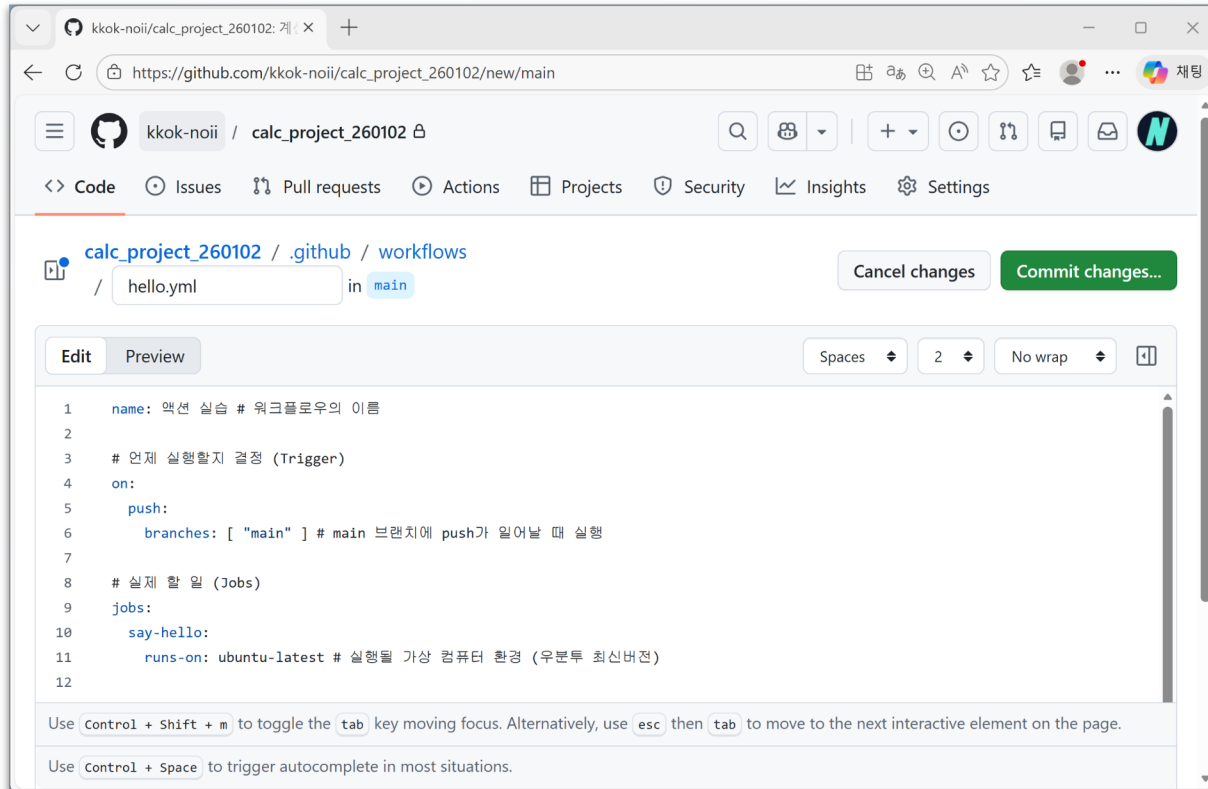
Action Script 작성해보기



Action

파일 경로에 .github/workflows/hello.yml

우측 내용 에디터에 붙여 넣기



name: 첫 번째 액션 실습 # 워크플로우의 이름

언제 실행할지 결정 (Trigger)

on:

push:

branches: ["main"] # main 브랜치에 push가 일어날 때 실행

실제 할 일 (Jobs)

jobs:

say-hello:

runs-on: ubuntu-latest # 실행될 가상 컴퓨터 환경 (우분투 최신버전)

steps:

1. 간단한 한 줄 명령어 실행

- name: 인사하기

run: echo "안녕하세요! GitHub Actions가 성공적으로 실행되었습니다. 🎉"

2. 여러 줄의 명령어 실행

- name: 현재 환경 확인하기

run: |

echo "현재 실행 중인 운영체제는?"

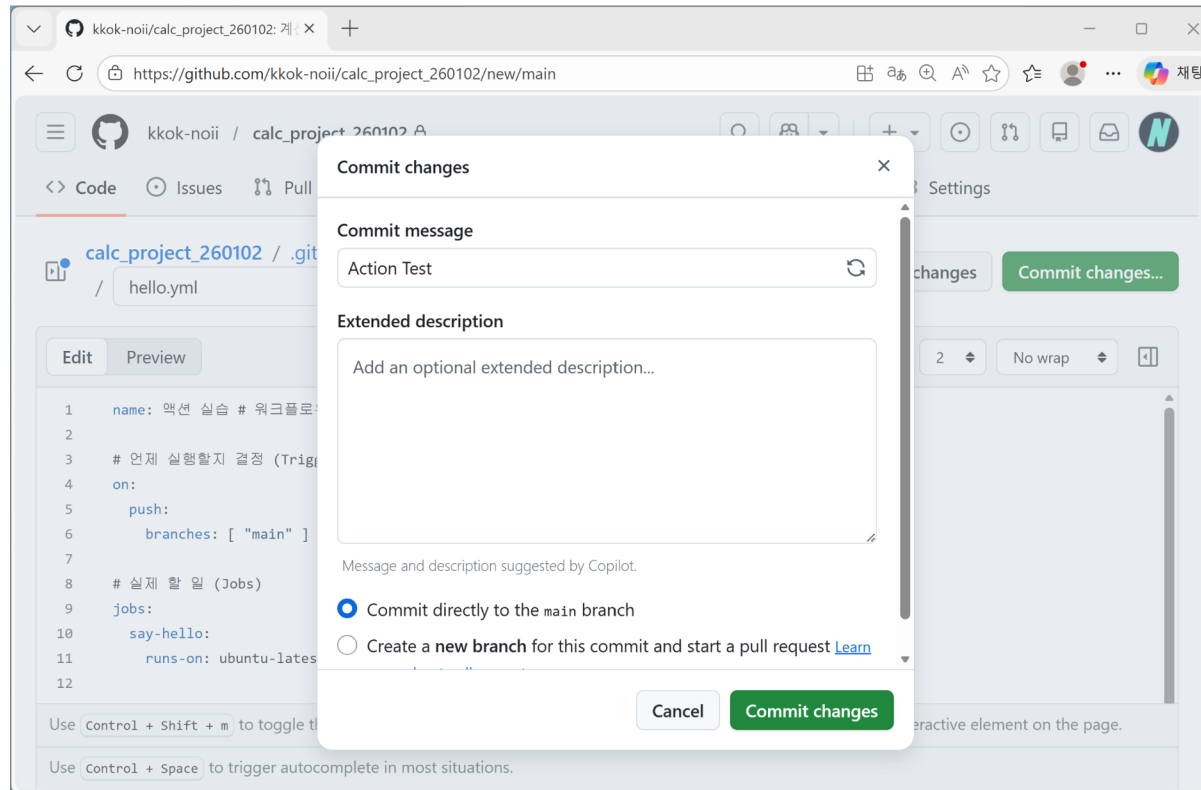
uname -a

echo "현재 디렉토리 파일 목록:"

ls -al

Action

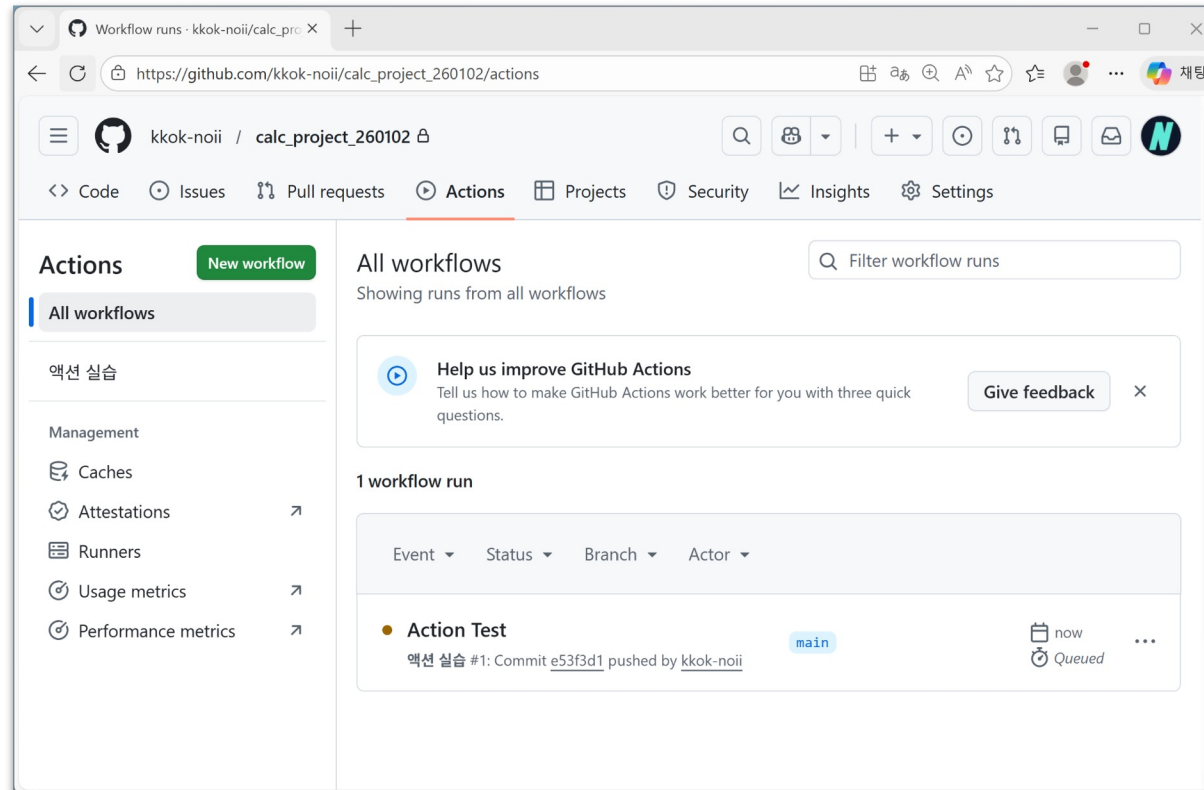
Github에서 바로 Commit 하기



Action

상단 Actions 탭에 들어가보면...

새로운 작업이 생겼습니다 (조금전에 commit을 했기에)



Action

Action이 수행되는 과정과 결과를 확인할 수 있습니다

성공 / 실패 여부도 확인 가능

The screenshot shows the GitHub Actions interface for a workflow named 'say-hello'. The top navigation bar includes links for Code, Issues, Pull requests, Actions (selected), Projects, Security, Insights, and Settings. The main content area displays the workflow run summary for 'say-hello', which is marked as 'Success' and completed in 8s. The workflow was triggered via a push to the 'main' branch. Below the summary, there is a section for 'hello.yml' showing the trigger 'on: push' and a list of jobs, including 'say-hello' which is also marked as successful.

https://github.com/kkok-noii/calc_project_260102/actions/runs/21029167974

kkok-noii / calc_project_260102

<> Code Issues Pull requests **Actions** Projects Security Insights Settings

← 액션 실행

✓ Action Test #1

Re-run all jobs

Summary

All jobs

say-hello

Run details

Usage

Workflow file

Triggered via push now

kkok-noii pushed · e53f3d1 · main

Status: Success

Total duration: 8s

Artifacts: -

hello.yml

on: push

say-hello 3s

The screenshot shows the detailed view of the 'say-hello' job. The top navigation bar is the same as the previous screenshot. The main content area displays the job details for 'say-hello', which is marked as 'succeeded 1 minute ago in 3s'. The job steps are listed, including 'Set up job', '인사하기', '현재 환경 확인하기', and 'Complete job'. The '인사하기' step shows a log entry: 'Run echo "안녕하세요! GitHub Actions가 성공적으로 실행되었습니다."'. The '현재 환경 확인하기' step shows a log entry: 'Run echo "현재 실행 중인 운영체제는?"'. The 'Complete job' step is also marked as successful.

https://github.com/kkok-noii/calc_project_260102/actions/runs/21029167974/job/60460762690

Summary

say-hello succeeded 1 minute ago in 3s

Search logs

All jobs

say-hello

Run details

Usage

Workflow file

Set up job 1s

24 Complete job name: say-hello

인사하기 0s

1 ▶ Run echo "안녕하세요! GitHub Actions가 성공적으로 실행되었습니다."

4 안녕하세요! GitHub Actions가 성공적으로 실행되었습니다.

현재 환경 확인하기 0s

1 ▶ Run echo "현재 실행 중인 운영체제는?"

7 현재 실행 중인 운영체제는?

8 Linux runner/m13qx 6.11.0-1018-azure #18~24.04.1-Ubuntu SMP Sat Jun 28 04:46:03 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux

9 현재 디렉토리 파일 목록:

10 total 8

11 drwxr-xr-x 2 runner runner 4096 Jan 15 11:08 .

12 drwxr-xr-x 3 runner runner 4096 Jan 15 11:08 ..

Complete job 0s